

Chapter

05

이번 장에서는 퀴즈 게임을 구현하며 AI가 만드는 콘텐츠의 정확성을 검증하는 방법을 배웁니다. 기본 게임 구조를 완성한 뒤 커스텀 명령어와 자동화 워크플로를 활용해 반복적인 작업을 효율적으로 관리하며 프로젝트를 확장해 나갑니다. 또한 매일 자동으로 업데이트되는 퀴즈 관리 시스템을 구축하며 AI를 진정한 개발 파트너로 활용합니다.

게임 제작으로 배우는 체계적인 개발과 관리

학습목표

- AI가 생성한 콘텐츠의 정확성을 검증하고 할루시네이션을 방지할 수 있습니다.
- 커스텀 명령어를 만들어 반복 작업을 자동화할 수 있습니다.
- 명령어 체이닝으로 여러 작업을 효율적으로 연결할 수 있습니다.
- 완성된 프로젝트에서 필요한 부분만 선택적으로 수정하고 업데이트할 수 있습니다.

05-1

할루시네이션 없는 AI 콘텐츠 만들기

핵심 키워드

할루시네이션

/memory

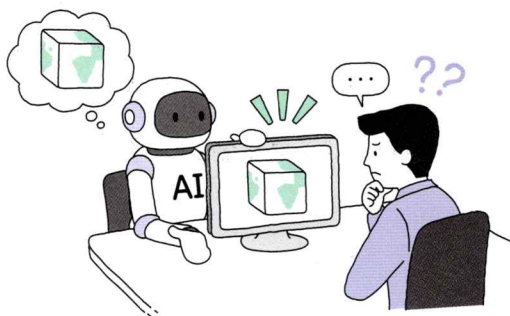
/clear

이번 절에서는 4지선다 상식 퀴즈 게임을 만들어 봅니다. 단순히 게임을 완성하는 데 그치지 않고, AI가 생성하는 문제의 정확성을 높이기 위한 방법까지 함께 익힐 것입니다. PRD 작성에서 구현까지 워크플로를 복습하고, 사실 확인을 위한 교차 검증 가이드라인 적용까지 과정을 따라가며 체계적인 개발과 관리의 기초를 다져 보겠습니다.

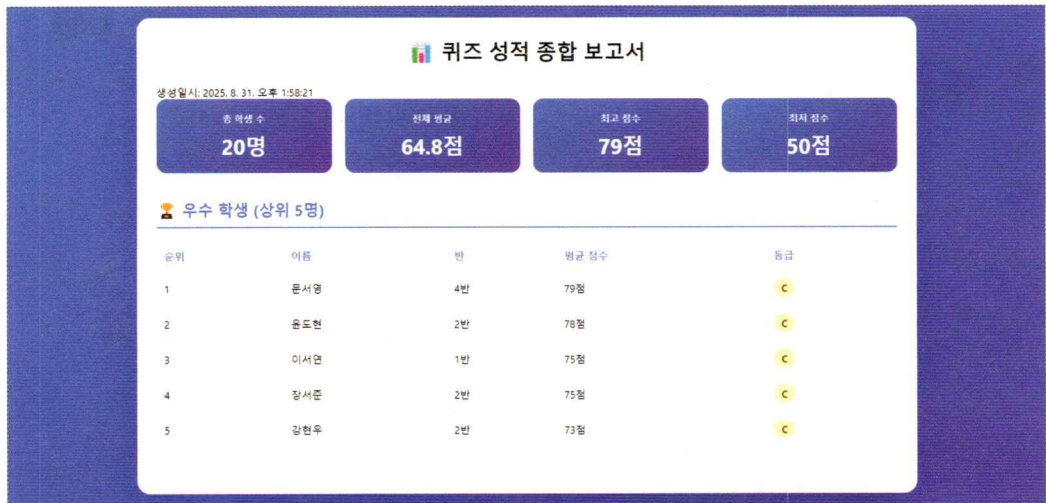
시작하기 전에

AI는 때때로 그럴듯하지만 틀린 정보를 생성합니다. 생성형 AI가 사실과 다른 내용을 진짜처럼 만들어내는 현상을 **할루시네이션** hallucination, 환각이라고 합니다. 원인은 AI가 학습 데이터에서 패턴을 찾아 이어 붙여 확률적으로 가장 그럴듯한 문장을 생성하기 때문이기도 하고, 불명확한 정보로 인한 추론 오류 때문이기도 합니다.

따라서 AI와 함께 일할 때는 할루시네이션 현상을 인식하고, 안전하게 검증하는 습관이 필요합니다. AI가 생성한 정보는 신뢰할 수 있는 문서 또는 공식 사이트와 비교해 교차 검증하거나, AI에게 '출처 없는 정보는 생성하지 말 것'과 같은 규칙을 입력해서 사전에 방지해야 합니다.



이번 절에서는 클로드와 클로드 코드를 활용해서 4지선다 상식 퀴즈 게임을 만들어 보겠습니다. 이 게임은 한국사, 과학, 지리, 예술과 문화 네 가지 분야에서 문제를 내고, 문제를 풀 때마다 정답 여부를 확인할 수 있습니다. 최종적으로는 점수와 순위가 기록되어 학습과 재미를 동시에 경험할 수 있죠.



1단계: 퀴즈 게임 구현하기

클로드에서 게임 제작을 위한 PRD를 작성하고 프롬프트를 준비한 뒤, 클로드 코드를 실행해서 게임을 제작할 것입니다. 그리고 나서 AI와 함께 제작한 게임이 기획 의도대로 잘 진행되는지 테스트해 보겠습니다.

그 과정에서 프로젝트 메모리에 검증 가이드라인을 저장하여 AI가 생성하는 콘텐츠의 정확성을 높이고, 대화 컨텍스트를 초기화하여 깨끗한 환경에서 테스트하는 방법을 익힐 수 있습니다.

미니 게임을 위한 PRD 작성하기

01 먼저 클로드(claude.ai)에 접속하여 상식 퀴즈 게임의 PRD를 작성해 봅시다. 다음과 같이 프롬프트를 입력하세요. 아티팩트에 PRD를 작성해 줍니다.

프롬프트

상식 퀴즈 게임을 만들려고 해. PRD를 작성해 줘.

게임 규칙:

- 4지선다 객관식 퀴즈
- 카테고리: 한국사, 과학, 지리, 일반상식

- 각 카테고리별 10문제씩 총 40문제
- 정답/오답 즉시 피드백
- 최종 점수 및 순위 기록

Korean general knowledge quiz game

+

🔔

📁

☰

H

상식 퀴즈 게임을 만들려고 해. PRD를 작성해 줘.

게임 규칙:

- 4지선다 객관식 퀴즈
- 카테고리: 한국사, 과학, 지리, 일반상식
- 각 카테고리별 10문제씩 총 40문제
- 정답/오답 즉시 피드백
- 최종 점수 및 순위 기록

상식 퀴즈 게임 PRD

문서

상식 퀴즈 게임의 PRD를 작성해드렸습니다. 주요 포인트들을 정리하면:

핵심 기능:

- 4지선다 객관식 40문제 (4: ↓ 퀘고리 × 10문제)
- 즉시 피드백 시스템으로 학습 효과 극대화

Claude에게 답변하기

H

+

≡

🔍

Sonnet 4

↑

☰

복사

게시

×

상식 퀴즈 게임 PRD (Product Requirements Document)

1. 프로젝트 개요

1.1 제품명

지식왕 도전 (Knowledge King Challenge)

1.2 제품 목적

다양한 분야의 상식을 재미있게 학습하고 테스트할 수 있는 4지선다 퀴즈 게임을 개발하여 사용자의 지식 습득과 두뇌 활동을 촉진한다.

1.3 타겟 사용자

- 연령: 중학생 이상 (14세+)
- 특징: 상식 퀴즈와 학습에 관심이 있는 사용자
- 이용 환경: 모바일/웹 브라우저

2. 핵심 기능 요구사항

note 2장에서 배운 5W1H 원칙을 떠올리며 요구사항을 정리해 보세요.

02 PRD를 클로드 코드용 프롬프트로 변환하고, 3단계 구현 가이드를 확인해 봅니다.

프롬프트

상식 퀴즈 게임을 클로드 코드로 구현할 수 있도록 PRD를 바탕으로 3단계 프롬프트를 정리해 줘.

Korean general knowledge quiz game

+

🔔

📁

☰

H

상식 퀴즈 게임을 클로드 코드로 구현할 수 있도록 PRD를 바탕으로 3단계 프롬프트를 정리해 줘.

상식 퀴즈 게임 개발 - Claude Code 3단계 프롬프트

문서

상식 퀴즈 게임을 Claude Code로 구현할 수 있는 3단계 프롬프트를 정리해드렸습니다.

각 단계별 특징:

Phase 1 (MVP):

- 핵심 게임 기능에 집중
- React + TypeScript 기반 구조
- 완전히 동작하는 최소 기능: ↓

Phase 2 (기능 개선):

Claude에게 답변하기

H

+

≡

🔍

Sonnet 4

↑

☰

복사

게시

×

상식 퀴즈 게임 개발 - Claude Code 3단계 프롬프트

Phase 1: 핵심 게임 기능 구현 (MVP)

```

bash

# Claude Code 실행이
claude code "상식 퀴즈 게임 MVP 구현"

```

프롬프트 1단계:

4지선다 상식 퀴즈 게임을 React + TypeScript로 구현해주세요.

****핵심 요구사항:****

1. ****게임 구조****

- 총 40문제 (한국사 10문제, 과학 10문제, 지리 10문제, 일반상식 10문제)
- 4지선다 객관식 형태

고급편

176 Chapter 05 | 게임 제작으로 배우는 체계적인 개발과 관리

퀴즈 게임 구현하기

01 데스크톱의 'VibeCoding' 폴더에 'Study-03' 폴더를 생성하고, 마우스 오른쪽 버튼을 누른 뒤 [터미널에서 열기]를 클릭해 명령 프롬프트를 엽니다. 그리고 나서 명령 프롬프트에 'claude'를 입력해 클로드 코드를 실행하세요.

클로드 코드를 실행하는 방법은 3장을 참고하세요.

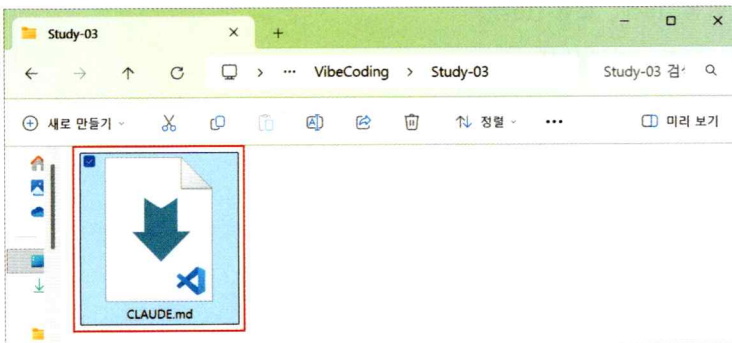


```
C:\Windows\System32\cmd
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user\Desktop\VibeCoding\Study-03>claude
```

02 '/init' 명령어를 입력해서 프로젝트를 초기화합니다. 'Study-03' 폴더에 'CLAUDE.md' 파일이 생성됩니다.

```
/init
```



03 본격적으로 미니 게임을 구현해 봅시다. 클로드 코드에 1단계 프롬프트를 입력합니다.

```
Quiz System
be made when development begins.

> 1단계: 핵심 퀴즈 시스템 구축
목표
기본적인 퀴즈 플레이가 가능한 MVP(Minimum Viable Product) 구현
구현 내용
1.1 프로젝트 초기 설정
- 프로젝트 구조 설정 (React 또는 Vanilla JS)
- 기본 HTML/CSS 레이아웃 구성
- 상태 관리 구조 설계
1.2 문제 데이터 구조 및 관리
javascript// 문제 데이터 구조 예시
{
  id: 1,
  category: "한국사",
```

note 클로드 코드 웹에서 복사한 뒤 붙여 넣으세요.

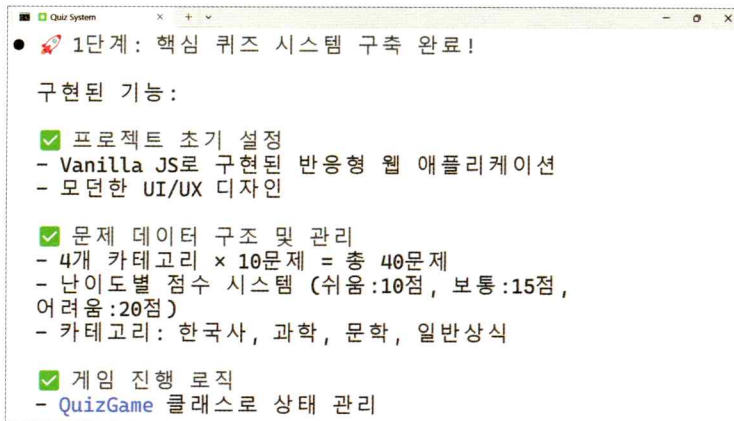
+ 여기서 잠깐

붙여 넣은 프롬프트가 너무 길 때

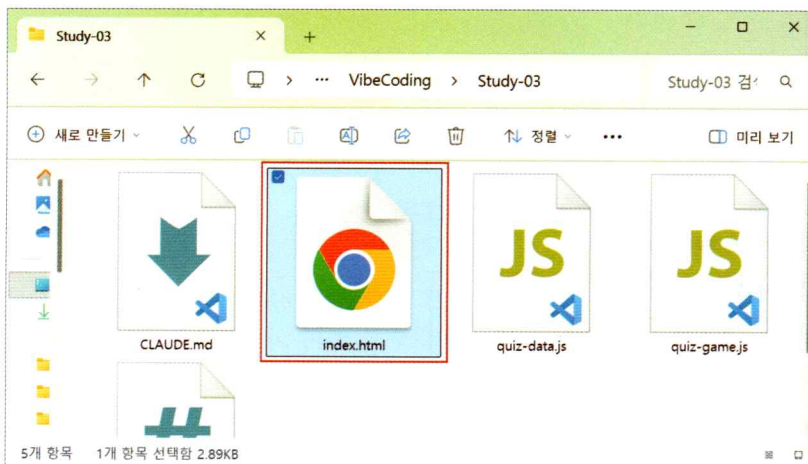
클로드가 제시한 프롬프트를 복사해서 클로드 코드에 붙여 넣을 때, 프롬프트가 너무 길면 '[Pasted text #1 +44 lines]'와 같이 내용이 요약되어 나타납니다. [Enter] 키를 눌러 실행하면 정상적으로 진행됩니다.

```
> [Pasted text #1 +44 lines]
> ▶ accept edits on (shift+tab to cycle)
```

04 퀴즈 시스템이 테스트 가능한 상태로 구현되었습니다.



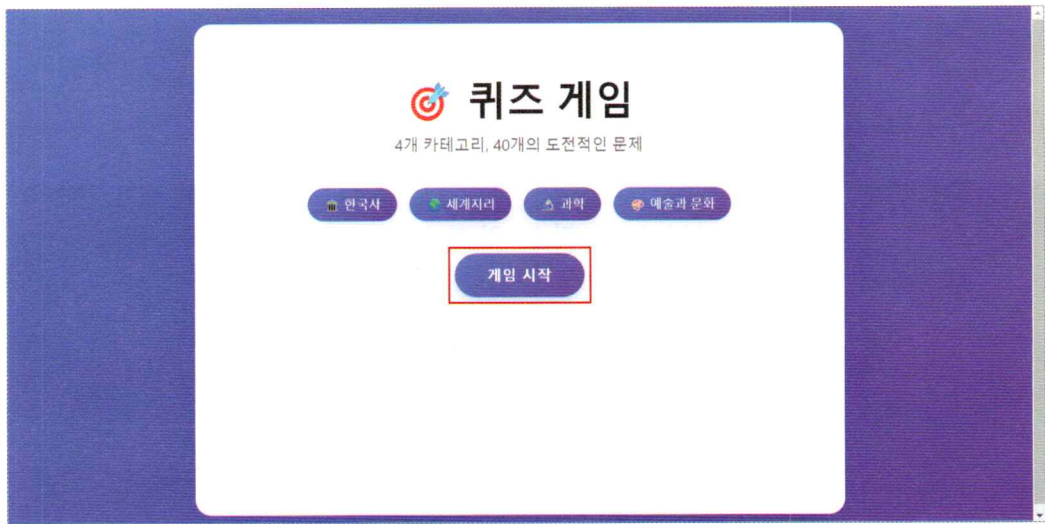
05 'Study-03' 폴더에 새로운 파일 'index.html'이 생성되었습니다.



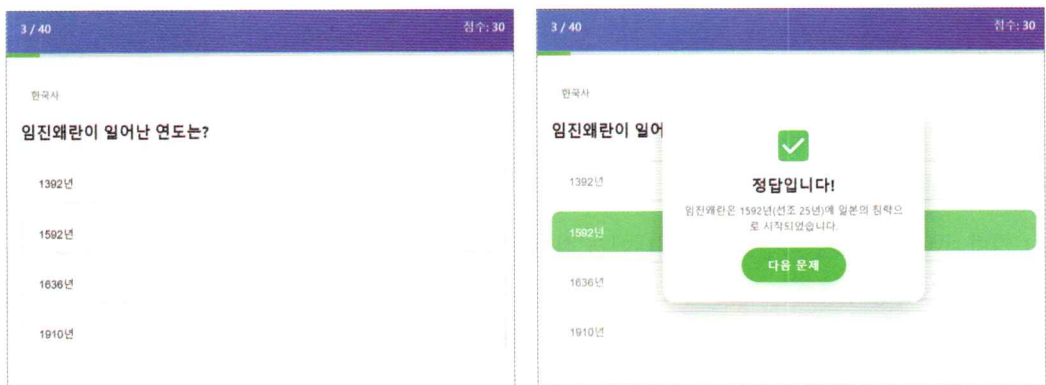
게임 테스트

제작한 상식 퀴즈 게임이 정상적으로 작동하는지 확인해 봅시다. 네 가지 카테고리의 문제들이 순차적으로 출제되는지, 정답과 오답 판정이 정확한지, 피드백이 즉시 표시되는지, 게임 완료 시 결과가 표시되는지를 테스트하면 됩니다.

01 모든 기능이 의도대로 작동하는지 확인하기 위해, 'Study-03' 폴더에서 'index.html' 파일을 더블 클릭해 웹 브라우저에서 상식 퀴즈 게임을 실행하고 [게임 시작] 버튼을 클릭한 뒤, 문제를 풀어 봅시다.



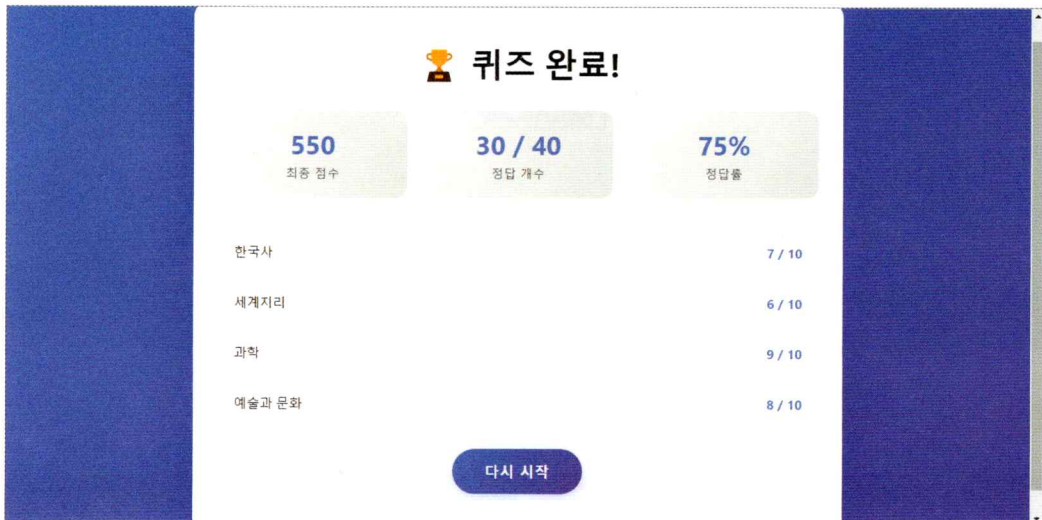
02 정답/오답 판정이 정확한지 확인하기 위해 문제를 풀어 보세요.



03 정답 여부를 판정하고, 결과를 즉시 피드백으로 제공합니다.



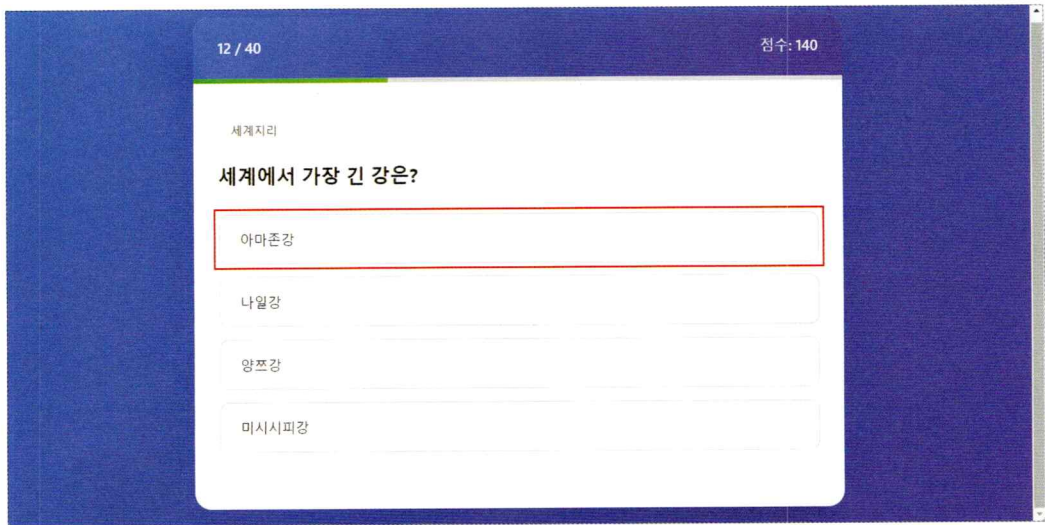
04 모든 퀴즈를 풀면 최종 점수를 보여줍니다.



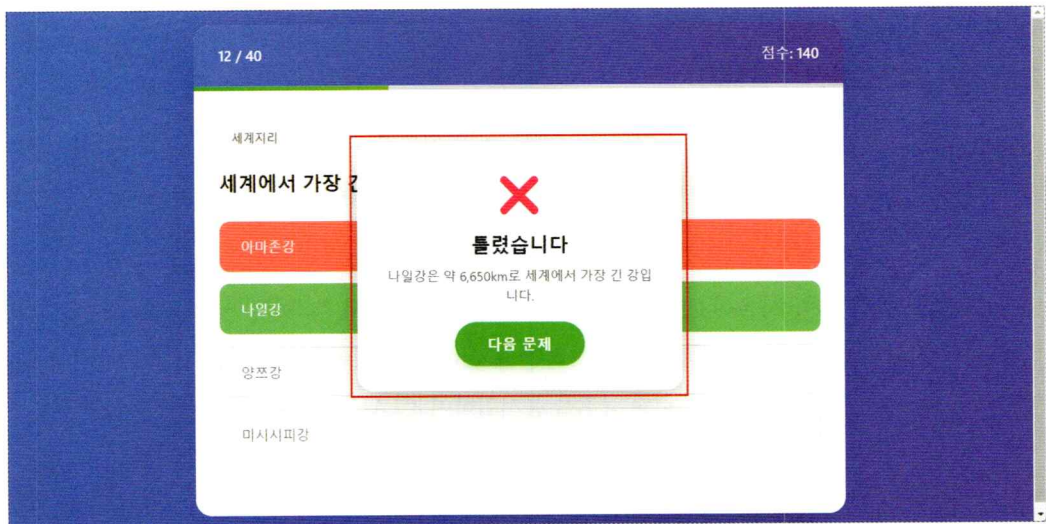
할루시네이션 해결하기

AI 기반 콘텐츠의 개발 과정은 할루시네이션 현상이 일어나지 않도록 검증하는 것이 중요합니다. 특히 정보의 정확성이 중요한 퀴즈 게임이라면 더욱더 그러하죠. 일단 지금 만들어진 문제들 중 명백한 오류는 없어 보이지만, 애매한 경우가 있을 수 있습니다.

다음은 '세계에서 가장 긴 강'이 무엇인지 묻는 퀴즈입니다. '아마존강'을 선택해 보겠습니다.



틀렸네요. 정답은 나일강입니다.



그런데 최근 연구에서 아마존강의 수원지를 재측정한 결과, 아마존강이 나일강보다 더 길다는 주장도 제기되고 있습니다. 이처럼 논란의 여지가 있는 주제는 문제 제작 시 교차 검증을 거쳐 명확한 기준을 세워야 합니다. 따라서 논란 있는 정보의 검증을 위해 검증 가이드라인을 마련하고, 이를 CLAUDE.md 파일에 영구적으로 저장하는 것이 가장 효과적입니다.

생성형 AI의 출력은 매번 임의적으로 달라지기 때문에, 여러분이 직접 실습할 때는 책에 나온 결과와 같은 오류가 재현되지 않을 수도 있습니다.

이번 장에서는 AI가 논란의 여지가 있거나 잘못된 정보를 만들어내는 현상을 이해하고 대처하기 위해, 의도적으로 논란의 여지가 있는 문제가 포함된 프로젝트의 라이브 데모를 제공합니다.

해당 자료는 책의 깃허브 저장소(<https://github.com/taehojo/vibecoding>) 하단의 [Chapter 5: 상식 퀴즈 게임] - [Study-03-basic(기본): 다운로드(실습 참조용)] 폴더에서 내려받을 수 있습니다. 이 파일을 참고하면 AI가 잘못된 출력을 만들어내는 과정을 직접 비교·분석하며 학습할 수 있습니다.

README

📁 프로젝트

📖

각 Chapter의 완성된 실습 프로젝트입니다. 라이브 데모를 클릭하여 바로 실행해보세요!

Chapter 3: 손글씨 인식 프로그램

• Study-01: 라이브 데모

📖

Chapter 4: 할 일 관리 앱

• Study-02: 라이브 데모

📖

Chapter 5: 상식 퀴즈 게임

• Study-03-basic (기본): 다운로드 (실습 참조용)

• Study-03-advanced (고급): 라이브 데모

📖

Chapter 6: 냉장고를 부탁해 (AI 레시피 추천)

• Study-04: 라이브 데모

📖

Chapter 7: 공감 AI 다이어리, PDF 문서 요약 AI

• Study-05 (공감 AI 다이어리): 라이브 데모

깃허브 저장소



교차 검증 가이드라인 구축하기

`/memory` 명령어는 AI가 기억하고 있는 프로젝트 관련 정보를 확인하고 관리합니다. 간단히 말하자면 클로드가 지금까지 기억하고 있는 내용을 확인하고 필요 시 수정하거나 삭제할 수 있는 기능입니다.

앞서 3장에서 '#' 기호를 사용해 CLAUDE.md에 짧은 규칙을 즉시 추가하는 방법을 살펴보았습니다. 하지만 보다 체계적으로 가이드라인을 편집하려면 `/memory` 명령어를 활용하는 것이 좋습니다. 지금부터 `/memory` 명령어를 사용해 검증 규칙을 문서화하여 'CLAUDE.md'에 저장해 보겠습니다.

01 'CLAUDE.md' 파일을 수정하기 위해, 클로드 코드에 `/memory` 명령어를 입력합니다.

/memory

> /memory

/memory

Edit Claude memory files ← 클로드의 메모리 파일을 수정합니다.

고급편

182 Chapter 05 | 게임 제작으로 배우는 체계적인 개발과 관리

02 메모리 파일을 저장할 위치를 지정해야 합니다. 퀴즈 게임의 검증 가이드라인은 현재 프로젝트에만 해당하므로 '1. Project memory'를 선택합니다.

화면 해설

메모리 저장 위치를 고르세요:

1. 프로젝트 메모리: 현재 프로젝트 폴더 안에서만 적용되는 규칙 저장
2. 사용자 메모리: 모든 프로젝트에 공통적으로 적용되는 사용자 전역 규칙 저장

```
> /memory
```

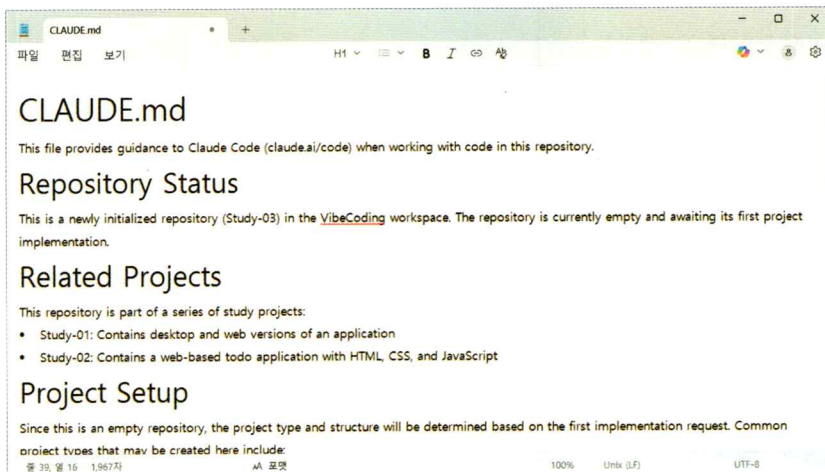
Learn more:

<https://docs.claude.com/en/docs/claude-code/memory>

Select memory to edit:

- ```
> 1. Project memory Saved in ./CLAUDE.md
 2. User memory Saved in ~/.claude/CLAUDE.md
```

**03** 현재 프로젝트의 CLAUDE.md 파일이 열립니다.



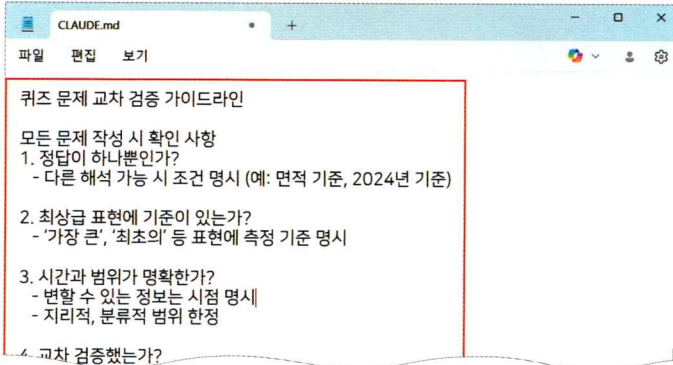
**note** 이 책에서는 시스템 에디터인 메모장을 사용합니다.

## 04 'CLAUDE.md' 파일의 마지막 행에 다음과 같이 교차 검증 가이드라인을 추가합니다.

### ▶ 퀴즈 문제 교차 검증 가이드라인

모든 문제 작성 시 확인 사항

1. 정답이 하나뿐인가?
  - 다른 해석 가능 시 조건 명시 (예: 면적 기준, 2024년 기준)
2. 최상급 표현에 기준이 있는가?
  - '가장 큰', '최초의' 등 표현에 측정 기준 명시
3. 시간과 범위가 명확한가?
  - 변할 수 있는 정보는 시점 명시
  - 지리적, 분류적 범위 한정
4. 교차 검증했는가?
  - 의심스러운 정보는 2개 이상 출처 확인
  - 논란 있는 내용은 주류 학설 기준



### + 여기서 잠깐 CLAUDE.md 작성법

CLAUDE.md에는 형식이 정해져 있지 않으므로, 간결하고 사람이 읽기 쉽게 작성하는 것이 핵심입니다. 또한 클로드의 프롬프트 일부로 작동하기 때문에 자주 쓰는 프롬프트처럼 지속적으로 다듬고 개선하는 것이 좋습니다.

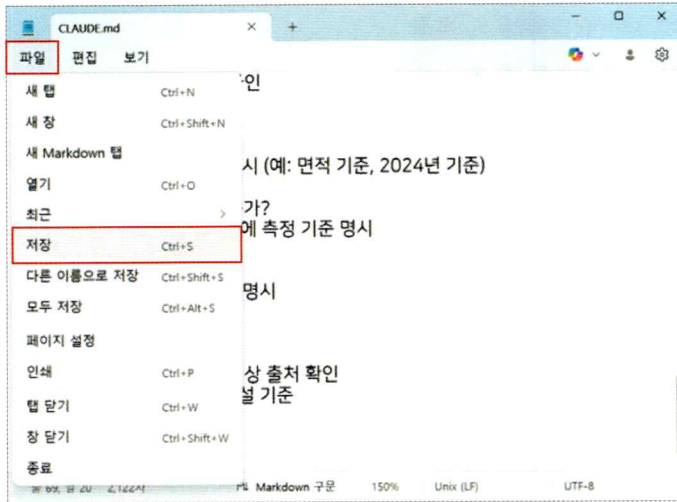
#### • 읽기 쉬운 자연어로 간결하게 작성하세요.

- 예: # 프로젝트 개요
- 이름: My Web App
  - 목표: 간단한 블로그 서비스 구현
  - 언어: Python 3.12, JavaScript(React)

#### • 중요한 규칙은 '중요', '반드시' 같은 강조 표현을 추가해 AI가 더 잘 따르도록 할 수 있습니다.

- 예: # 주의사항
- settings.json에 실제 API 키 입력 금지 → 반드시 .env 파일 사용
  - 이미지 파일은 /assets/images 폴더에만 저장

**05** [파일]-[저장(**Ctrl**+**S**)] 메뉴를 클릭해 CLAUDE.md 파일을 저장하고, 텍스트 에디터를 닫습니다.



**06** 클로드 코드로 돌아오면 CLAUDE.md 파일이 열렸었다는 메시지를 볼 수 있습니다.

#### 화면 해설

```
> /memory 실행 결과
└ 열린 메모리 파일: ./CLAUDE.md
 > 다른 편집기를 사용하려면 $EDITOR 또는 $VISUAL 환경 변수를 설정하세요
```

```
> /memory
└ Opened memory file at ./CLAUDE.md

 > To use a different editor, set the $EDITOR or $VISUAL
 environment variable.
```

**07** 수정된 사항이 반영되었는지 확인하기 위해 다음과 같이 프롬프트를 입력합니다.

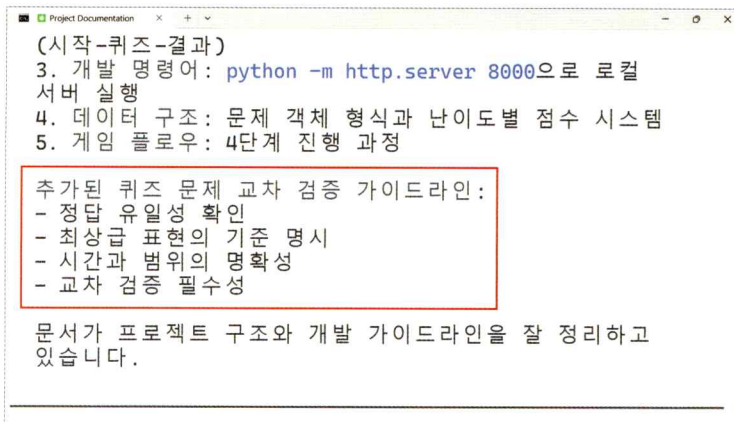
#### 프롬프트

현재 프로젝트의 CLAUDE.md 내용을 확인해 줘.

```
> 현재 프로젝트의 CLAUDE.md 내용을 확인해 줘. █
┌──┐
│ ▶ accept edits on (shift+tab to cycle) ·1 background task cycle │
└──┘
```



**08** 방금 추가한 가이드라인을 포함한 CLAUDE.md의 전체 내용을 확인할 수 있습니다.



## 컨텍스트 초기화하기

`/clear` 명령어는 현재 세션의 대화와 작업 컨텍스트를 모두 지우고 초기화합니다. 대화 기록이 너무 길어져서 작업에 불필요한 컨텍스트가 개입할 때, 이전 프로젝트의 흔적을 지우고 완전히 새로운 작업을 시작할 때, 테스트 도중 잘못된 컨텍스트 때문에 혼선이 있을 때 사용할 수 있습니다.

이제 새롭게 추가된 가이드라인대로 클로드 코드가 작동하는지 테스트해 볼 텐데, 이전 대화에서 생성했던 문제로부터 영향을 받을 수 있습니다. 중요한 규칙은 CLAUDE.md 파일에 저장해 두었으므로, `/clear` 명령어를 활용해 지금까지 대화 내용과 작업 정보를 모두 지우고 상식 퀴즈 게임을 재실행해 봅시다.

### + 여기서 잠깐 /clear와 /compact의 차이

`/clear`와 마찬가지로 앞서 배운 `/compact`도 컨텍스트를 정리하고 최적화하는 기능을 수행합니다. 하지만, 대화를 요약하여 유지하는 `/compact`와 달리 `/clear`는 대화를 완전히 초기화한다는 차이가 있습니다. 다음은 `/clear`와 `/compact`의 차이를 정리한 테이블입니다.

| 구분           | <code>/clear</code>                                     | <code>/compact</code>                                    |
|--------------|---------------------------------------------------------|----------------------------------------------------------|
| 기본 동작        | 대화 기록을 완전히 초기화                                          | 대화 내용을 요약하여 압축                                           |
| 컨텍스트 윈도우     | 완전히 비움                                                  | 요약된 형태로 유지                                               |
| CLAUDE.md 파일 | 유지됨                                                     | 유지됨                                                      |
| 사용 시점        | • 새 작업/기능 시작<br>• 완전히 다른 주제로 전환<br>• 잘못된 방향으로 진행된 대화 리셋 | • 컨텍스트 창이 가득 찰 때<br>• 같은 작업 계속하면서 최적화 필요<br>• 토큰 사용량 줄이기 |

**01** 작업 내용을 초기화하기 위해 클로드 코드에 '/clear'를 입력합니다.

```
> /clear
```

```
> /clear
/clear (reset, new)
Clear conversation history and free up context ← 대화 기록을 삭제하고 컨텍스트를 비웁니다.
```

**02** 모든 대화 기록이 사라지는 것을 확인합니다.

```
> /clear
L (no content)
```

```
>
```

```
▶▶ accept edits on (shift+ta ·1 background ·? for
to cycle) task shortcuts
```

## 가이드라인 점검하기

**01** 클로드 코드에 다음과 같은 프롬프트를 입력합니다. 새로운 가이드라인에 따라 문제가 만들어졌다고 안내됩니다.

### 프롬프트

지금까지 만든 문제를 방금 저장한 가이드라인을 참고해서 점검해 줘. 가이드라인에 맞지 않은 퀴즈와 정답이 있다면 이에 맞게 수정해 줘.

```
● 가이드라인에 따른 퀴즈 문제 점검 및 수정을 완료했습니다.
```

수정된 내용 :

1. 문제 35: "세계에서 가장 큰 대륙은?" → "세계에서 면적이 가장 큰 대륙은?"  
- 측정 기준(면적)을 명시하여 답의 명확성 확보
2. 문제 347: "세계에서 가장 높은 산은?" → "세계에서 해발고도가 가장 높은 산은?"  
- 측정 기준(해발고도)을 명시하고 정확한 수치(8,848m) 추가

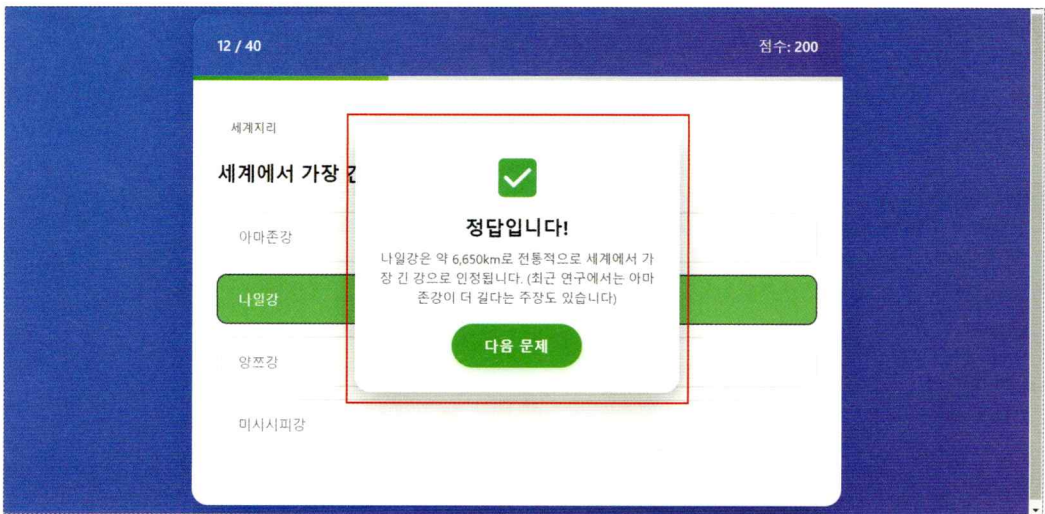
점검 결과 양호한 문제들 :

- 대부분의 문제들이 정답이 하나뿐이고 명확함

**02** 다시 'index.html' 파일을 실행해 퀴즈에 가이드라인이 반영되었는지 확인해 보겠습니다. 문제에 '전통적 측정 기준'이라는 조건이 표시됩니다. '나일강'을 선택해 보겠습니다.



**03** 정답 표시와 함께 추가 정보도 안내합니다.





이번 절에서는 퀴즈 게임을 제작하며 AI가 생성하는 콘텐츠의 정확성을 관리하는 방법을 배웠습니다. 프로젝트의 CLAUDE.md 파일에 교차 검증 가이드라인을 저장하여 AI 할루시네이션을 방지하는 체계를 마련했습니다. 그 과정에서 /clear 명령어로 작업 맥락을 초기화하여 가이드라인이 제대로 적용되는지 테스트하고, 애매한 표현에는 '전통적 측정 기준' 같은 명확한 조건을 추가했습니다.

다음 절에서는 프로젝트의 개발 효율을 높이기 위해 커스텀 명령어를 만들어 보겠습니다.

저자의 깃허브 자료실에서는  
프로젝트가 작동하는 모습을  
실시간으로 시연해 주는 라  
이브 데모를 제공합니다.

깃허브 자료실



## 마무리

### ▶ 3가지 키워드로 끝내는 핵심 포인트

- AI가 그럴듯하지만 사실과 다른 정보를 진짜처럼 만들어내는 현상을 **할루시네이션**이라고 합니다.
- **/memory**는 AI가 기억하는 프로젝트 관련 정보를 확인하고 관리하는 명령어입니다.
- **/clear**는 현재 세션의 대화와 작업 맥락을 모두 지우고 초기화하는 명령어입니다.

### ▶ 확인 문제

1. 다음 설명 중 틀린 것을 고르세요.

- ① AI 할루시네이션은 패턴을 과도하게 일반화할 때 발생할 수 있다.
- ② **/memory** 명령어에서 Project memory를 선택하면 **./CLAUDE.md** 파일이 열린다.
- ③ **/clear**는 대화를 요약하여 압축하고 컨텍스트를 유지한다.
- ④ **./CLAUDE.md**에 가이드라인을 추가할 때는 자연어로 간결하고 명확하게 작성하면 된다.

2. 다음 상황에서 사용할 적절한 명령어나 대응 방법을 쓰세요.

| 상황                              | 대응 |
|---------------------------------|----|
| 퀴즈 게임의 검증 가이드라인을 저장하고 싶을 때      |    |
| 이전 대화가 새 작업에 영향을 줄 것 같을 때       |    |
| 대화가 길어져서 토큰을 절약하고 싶을 때          |    |
| "세계에서 가장 긴 강" 같은 애매한 문제를 발견했을 때 |    |

# 05-2

## 자동화로 개발 효율 높이기

핵심 키워드

커스텀 명령어

전역 명령어

프로젝트 명령어

인수

플레이스홀더

퀴즈 게임의 기본 기능을 완성했으니, 이제 퀴즈를 검증하고 분석해서 품질을 높이고 새 문제를 추가하는 등 콘텐츠를 확장해 봅시다. 이 절에서는 반복적인 작업을 줄여 개발 효율을 높이기 위해 클로드 코드의 커스텀 명령어를 활용할 것입니다. 이를 통해 프로젝트를 더 빠르고 안정적으로 관리하며, AI 도구를 개발 워크플로에 자연스럽게 통합하는 경험을 하게 됩니다.

### 시작하기 전에

우리는 앞서 1장에서 웹사이트의 디자인을 **커스터마이즈** customize 해 보았습니다. 커스터마이즈는 프로그램의 시스템이 제공하는 기본 기능을 그대로 쓰지 않고 사용자가 원하는 방식으로 설정, 모양, 동작을 조정하는 것을 의미하죠.

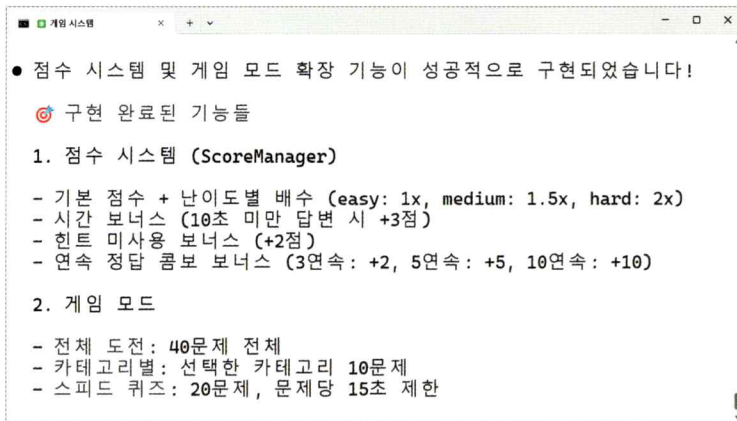
클로드 코드의 슬래시 명령어도 사용자가 명령어의 동작이나 형태를 바꾸어 새로 정의할 수 있습니다. 이를 통해서 반복적인 일을 대신 처리해 주는 자동 실행 명령어를 만들 수 있죠.



## 2단계: 게임 확장하기

이제 기본 퀴즈 시스템이 완성되었으니, 게임을 더욱 흥미롭게 만들기 위한 점수 시스템과 다양한 게임 모드를 추가해 보겠습니다.

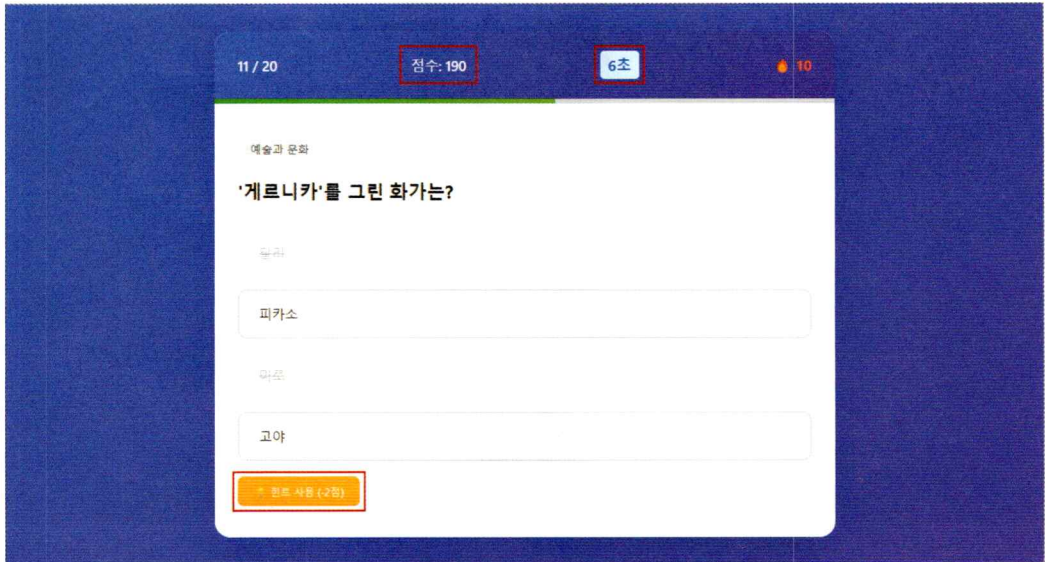
**01** 클로드 코드에 앞서 만든 2단계 프롬프트를 입력합니다. 실행을 마치면 완료되었다는 메시지가 나타납니다.



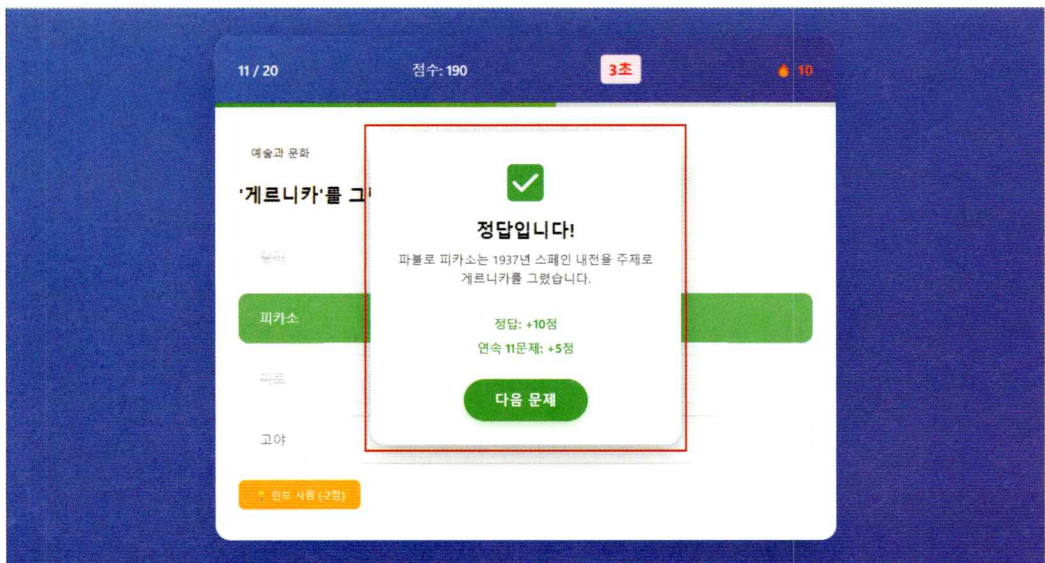
**02** 'Study-03' 폴더에서 'index.html' 파일을 더블 클릭해 게임을 실행하세요. 게임 모드 세 가지가 추가된 것을 확인할 수 있습니다. 이 중 [스피드 퀴즈]를 선택해 보겠습니다.



**03** 제한 시간 15초 룰이 적용되었습니다. [힌트 사용] 버튼을 누르면 힌트를 확인할 수 있고, 점수가 차감됩니다.



**04** 정답/오답을 잘 판별해 줍니다.



**05** 퀴즈를 다 풀면 최종 점수를 확인할 수 있습니다.



## 커스텀 명령어로 반복 작업 자동화하기

퀴즈 게임은 교육 콘텐츠나 학습용 앱으로도 활용할 수 있습니다. 하지만 이런 프로그램을 개발하다 보면 비슷한 작업을 반복해야 하는 경우가 많습니다. 예를 들어 문제의 형식을 통일하기 위해 복사·붙여넣기를 반복하거나, 문제를 검증할 때마다 가이드라인을 확인하고, 균형을 맞추기 위해 난이도를 조정하는 등 작업이 꾸준히 따라오죠. 모두 개발 과정에서 흔히 발생하는 반복 작업의 대표적인 예입니다.

이런 반복 작업을 매번 긴 프롬프트로 처리하려고 하면 효율이 떨어질 뿐 아니라 실수도 잦아집니다. 이럴 때 유용한 기능이 바로 클로드 코드의 커스텀 명령어입니다. 커스텀 명령어를 활용하면 반복 작업을 간단한 명령어로 자동화할 수 있어 빠르고 안정적으로 개발을 이어갈 수 있습니다.

### 커스텀 명령어

**커스텀 명령어** custom command란 사용자가 직접 정의해서 쓸 수 있는 나만의 단축 지시어입니다. 클로드 코드가 기본으로 제공하는 명령어 외에도 반복적으로 쓰는 지시사항을 커스텀 명령어로 만들어 두면 매번 긴 프롬프트를 다시 입력하지 않아도 됩니다. 명령어는 마크다운 markdown 파일로 정의되며, 범위에 따라 다음 두 종류로 나뉩니다.

## 1. 전역 명령어

- 홈 디렉터리 위치에서 정의해 모든 프로젝트에 공통으로 적용되는 명령어입니다.
- 위치: ~/.claude/commands/ ← 터미널 경로 표기법에서 '~'는 홈 디렉터를 뜻합니다.

## 2. 프로젝트 명령어

- 프로젝트 디렉터리 위치에서 정의해 현재 작업 중인 프로젝트에서만 쓸 수 있는 명령어입니다.
- 위치: .claude/commands/ ← 터미널 경로 표기법에서 '.'는 현재 디렉터를 뜻합니다.

## 명령어 폴더 생성하기

실습을 통해 간단한 커스텀 명령어를 만들어 봅시다. 이 커스텀 명령어는 상식 퀴즈 게임 프로젝트에만 적용할 것이므로, 'Study-03' 폴더에 명령어를 저장할 폴더를 새로 생성할 것입니다.

데스크톱의 'Study-03' 폴더 위치에서 명령 프롬프트를 열고, 클로드 코드를 실행해 준비하고, 다음과 같이 입력하세요. 클로드 코드가 명령어를 저장할 폴더를 생성하고 디렉터리 구조를 보여줍니다.

### 프롬프트

현재 프로젝트에 커스텀 명령어 폴더를 만들어 줘.  
.claude/commands 디렉터를 생성하고 구조를 보여줘.

- ☒ .claude/commands 디렉터리가 성공적으로 생성되었습니다!

📁 생성된 디렉터리 구조

```
C:\Users\user\Desktop\VibeCoding\Study-03\
├── .claude/
│ └── commands/ ← .claude/commands 디렉터리
├── index.html
├── styles.css
├── quiz-data.js
├── quiz-game.js
└── CLAUDE.md
```

.claude/commands 폴더는 Claude Code에서 커스텀 슬래시 명령어를 정의하는 데 사용됩니다. 이 폴더에 JavaScript 파일을 추가하여 프로젝트별 맞춤 명령어를 만들 수 있습니다.

## 첫 번째 커스텀 명령어 만들기

상식 퀴즈 게임의 CLAUDE.md 파일에 입력했던 '2. 최상급 표현에 기준이 있는가?' 항목은 AI의 할루시네이션 현상을 방지하기 위해 최상급 표현에 대해 측정 기준을 명시할 것을 요구하는 교차 검증 가이드라인이었습니다.



이제 클로드 코드에 퀴즈의 품질을 점검하기 위한 커스텀 명령어를 등록하고, 재시작 후 실행해 모호한 최상급 표현을 찾아낸 뒤, 고치게 해 봅시다.

**01** 'commands' 폴더 내에 첫 번째 커스텀 명령어를 만들어 보겠습니다. 클로드 코드에 다음과 같이 입력하세요.

#### 프롬프트

.claude/commands/quiz-validate.md 파일을 만들어 줘.

퀴즈 문제 중에서 '가장', '최초', '최대' 같은 최상급 표현이 있는지 찾아서 목록으로 보여줘.

```
> .claude/commands/quiz-validate.md 파일을 만들어 줘.
퀴즈 문제 중에서 '가장', '최초', '최대' 같은 최상급 표현이 있는지
찾아서 목록으로 보여줘
```

▶ accept edits on (shift+tab to cycle) · 2 background tasks

**02** 논란의 소지가 있는 답변을 방지하기 위해, 문제 중 최상급 표현을 찾아서 보여주는 커스텀 명령어 '/quiz-validate'를 생성했습니다.

#### 세부 분석

1. "첫 번째" - 순서 기준으로 명확
2. "가장 가까운" - 거리 기준으로 명확
3. "최초로" - 연도(1869년) 명시로 시간 기준 명확
4. "최초의" - 한국 문학사 기준으로 명확
5. "면적이 가장 큰" - 면적 기준 명시 (수정됨)
6. "해발고도가 가장 높은" - 해발고도 기준 명시 (수정됨)

#### 결론

모든 최상급 표현이 포함된 문제들이 가이드라인을 완벽히 준수하고 있습니다. 특히 35번과 38번 문제는 이전 점검에서 수정되어 측정 기준이 명확히 명시되었습니다.

quiz-validate.md 커스텀 명령어 파일이 생성되었으며, 향후 /quiz-validate 명령어로 이러한 검증을 자동화할 수 있습니다.

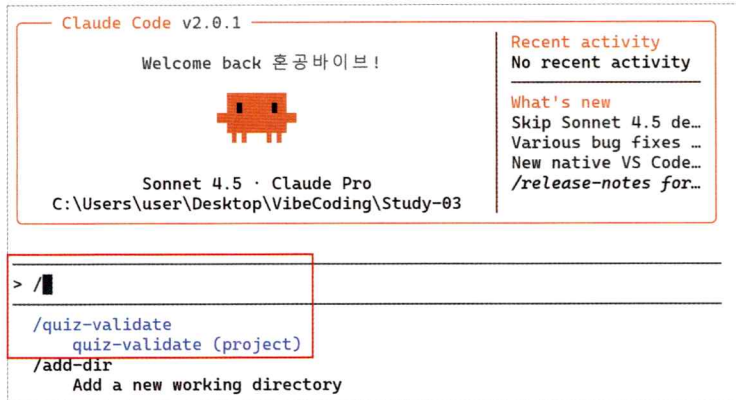
**03** 커스텀 명령어를 실행하기 위해서는 클로드 코드를 종료하고 재실행해야 합니다. 클로드 코드에 '/exit' 명령어를 입력해 종료하겠습니다.

▶ /exit

```
> /exit
/exit (quit)
Exit the REPL
```

**note** **Ctrl**+**C** 키를 두 번 눌러도 클로드 코드를 종료할 수 있습니다.

**04** 명령 프롬프트에 'claude'를 입력해 클로드를 재시작합니다. '/(슬래시)'를 입력하면 커스텀 명령어 '/quiz-validate'가 추가된 것을 확인할 수 있습니다.



**note** '/help' 명령어를 입력하면 등록된 모든 명령어를 볼 수 있습니다.

**05** 클로드 코드에 '/quiz-validate'를 입력해 실행하겠습니다. 퀴즈에서 최상급 표현을 골라내고 개선 방안을 제안합니다.



새로 생성한 커스텀 슬래시 명령어를 활용하면, 상식 퀴즈 게임뿐 아니라 이와 비슷한 다른 프로젝트를 진행할 때도 AI가 생성한 사실을 간단히 교차 검증하고 부정확하거나 논란의 여지가 있는 문제를 자동으로 방지할 수 있습니다.

## 커스텀 명령어 확장하기

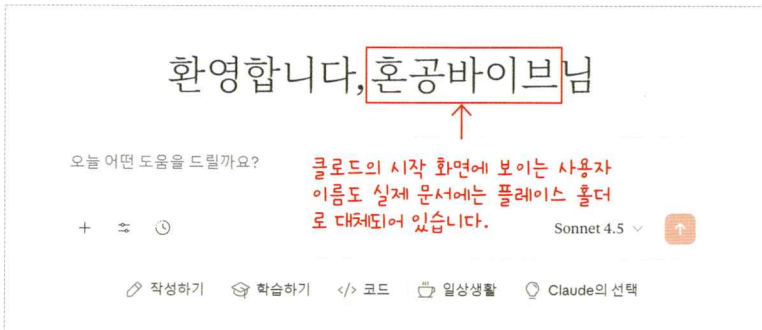
**인수** argument 는 명령어나 함수에 전달하는 추가 정보로 명령어 command 가 ‘무엇을 해야 할지’를 정한다면, 인수는 ‘무엇을 대상으로 어떻게 할 것인가’를 구체적으로 알려주는 값입니다.

지금부터 할 일은 퀴즈의 품질과 정확성을 검증하는 커스텀 명령어 ‘/quiz-validate’에 **인수 플레이스홀더** placeholder 를 지정해서 두 가지 방식으로 확장하는 것입니다. 첫 번째는 지정한 카테고리의 퀴즈만 검증하도록 인수에 카테고리를 전달하는 것이고, 두 번째는 지정한 범위의 퀴즈만 검증하도록 인수에 두 숫자를 전달하는 것입니다.

- /quiz-validate **한국사** ← 특정 카테고리만 검증
- /quiz-range **1 20** ← 1~20번 문제만 분석

### + 여기서 잠깐 **플레이스홀더**

**플레이스홀더** placeholder 는 나중에 실제 값이 들어갈 자리를 표시하는 기호입니다. 쉽게 말하면 아직 정해지지 않은 값이 들어갈 자리에 빈칸을 미리 만들어 놓고, 실행할 때 그 자리에 실제 값이 들어가도록 하는 장치입니다. 예를 들어 웹사이트에서 ‘환영합니다, {이름}님’과 같이 ‘{이름}’이라는 플레이스홀더를 만들어 두면, 실제로 실행할 때 사용자의 이름이 플레이스홀더 자리에 들어가도록 하는 것입니다.



### 특정 카테고리만 검증하기

/quiz-validate 명령어가 사용자가 입력한 카테고리명을 인수로 받아서 해당 카테고리의 퀴즈만 검증하도록 확장해 봅시다. 다음과 같이 명령어에 카테고리를 받을 수 있도록 인수 플레이스홀더를 설정하고, 인수가 없으면 퀴즈 전체를 검증하도록 하겠습니다.

▶ /quiz-validate **\$ARGUMENTS** ← 인수 플레이스홀더

**01** 커스텀 명령어에 인수를 지정하기 위해 '.claude/commands/quiz-validate.md' 파일을 수정하겠습니다. 클로드 코드에 다음과 같이 프롬프트를 입력하세요.

**프롬프트**

.claude/commands/quiz-validate.md를 다음과 같이 수정해 줘.

사용자가 카테고리를 지정하면 해당 카테고리의 문제만 검증하고,  
지정하지 않으면 전체 문제를 검증해 줘.

\$ARGUMENTS에 입력된 값을 카테고리로 사용하면 돼.

검증할 때는 '가장', '최초', '최대' 같은 모호한 표현이 있는지 찾아서  
어떤 기준을 명시해야 하는지 알려 줘.

> .claude/commands/quiz-validate.md를 다음과 같이 수정해 줘.

사용자가 카테고리를 지정하면 해당 카테고리의 문제만 검증하고,  
지정하지 않으면 전체 문제를 검증해 줘.  
\$ARGUMENTS에 입력된 값을 카테고리로 사용하면 돼.

검증할 때는 '가장', '최초', '최대' 같은 애매한 표현이 있는지 찾아서  
어떤 기준을 명시해야 하는지 알려 줘. █

**02** 완료되었다는 메시지를 확인하면 '/exit' 명령어를 입력해서 클로드 코드를 종료합니다.

▶ /exit

> /exit █

/exit (quit)  
Exit the REPL

**03** 'claude'를 입력해 새로 시작한 뒤, '/quiz-validate 한국사' 명령어를 입력하세요. '한국사' 카테고리의 퀴즈만 검증하는 명령어입니다.

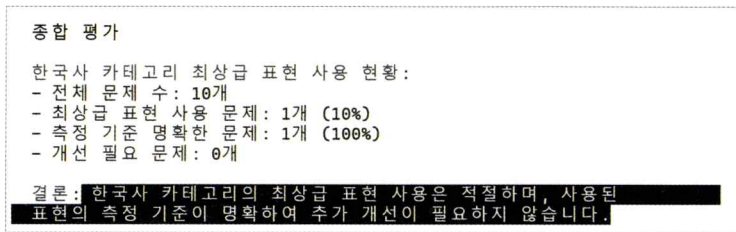
▶ /quiz-validate 한국사





**note** 중간 과정을 확인해 가면서 '1. Yes'를 눌러 끝까지 진행시킵니다.

## 04 '한국사' 카테고리의 퀴즈를 검증했습니다.



## 특정 범위만 분석하기

특정 범위의 퀴즈를 분석하는 커스텀 명령어 '/quiz-range'를 생성해 보겠습니다. 이 명령어는 두 개의 위치 인수를 받아 검토할 퀴즈의 범위를 지정합니다. 예를 들어, '/quiz-range 1 20'는 1번부터 20번까지 퀴즈를 검증합니다.

이 명령어는 퀴즈의 난이도와 정답 분포를 중점적으로 분석합니다. 지정된 구간에 포함된 퀴즈의 난이도 평균과 편차를 계산하여, 너무 쉽거나 어려운 문제가 특정 구간에 몰려 있지는 않은지 확인합니다.

또 정답이 특정 보기에 치우쳐 있는지도 함께 점검합니다. 예를 들어, 정답이 지나치게 1번에만 몰려 있거나, 4번 보기는 거의 정답으로 사용되지 않는 상황을 찾아냅니다. 이러한 분석 결과를 바탕으로 전체 퀴즈의 밸런스를 고르게 유지할 수 있도록 방향을 제시해 줍니다.

▶ /quiz-range \$시작 번호 \$끝 번호 ← 인수 플래시홀더

**01** 특정 범위의 퀴즈를 검토하는 커스텀 명령어 '/quiz-range'를 생성하기 위해 클로드 코드에 다음과 같이 프롬프트를 입력합니다.

#### 프롬프트

.claude/commands/quiz-range.md를 생성해.

\$1번부터 \$2번까지의 문제를 검토하는 기능을 만들어 줘.

문제의 난이도와 정답 분포를 확인하게 해 줘.

> .claude/commands/quiz-range.md를 생성해.

\$1번부터 \$2번까지의 문제를 검토하는 기능을 만들어 줘.  
문제의 난이도와 정답 분포를 확인하게 해 줘.

\$1과 \$2는 각각 시작 번호와 끝 번호를 나타내는  
플레이스홀더로, 실제 명령어 실행 시 사용자가  
입력한 인수로 대체됩니다.

● I'll create a new slash command file for reviewing quiz questions within a specific range.

● Write(.claude\commands\quiz-range.md)

└ Wrote 1 lines to .claude\commands\quiz-range.md

---  
description: 지정한 범위의 퀴즈 문제를 검토하고 난이도와 정답 분포를 분석합니다

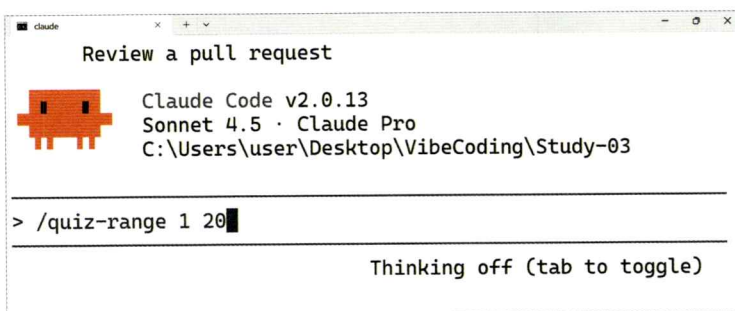
---  
quiz-data.js 파일을 읽고, \$1번부터 \$2번까지의 문제를 검토해 줘.

각 문제에 대해:

1. 문제 번호, 카테고리, 난이도
2. 문제 내용

**02** '/exit' 명령어를 입력해 클로드를 종료한 뒤, 'claude' 명령어로 재시작하세요. 그리고 나서 커스텀 명령어 '/quiz-range'가 잘 작동하는지 확인하기 위해 1번부터 20번까지 퀴즈를 검사하는 명령어를 다음과 같이 입력합니다.

▶ /quiz-range 1 20



**03** 명령어가 정확히 작동하여 1-20번 퀴즈를 분석합니다. 정답 위치를 균등하게 분산시켜서 편향을 완화할 것을 권장하네요.

---

#### ⚠ 경고 사항

1. 정답 편향 심각: 정답이 0번 선택지에 55% 집중되어 있습니다. 이는 수험자가 패턴을 파악하여 전략적으로 답할 수 있는 문제입니다.
2. 3번 선택지 미사용: 3번 선택지가 단 한 번도 정답으로 사용되지 않았습니다. 정답이 0번과 1번에만 집중되어 있습니다.
3. 권장 사항:
  - 정답 위치를 더 균등하게 분산시키세요 (각 25% 목표)
  - 특히 2번, 3번 선택지를 정답으로 더 많이 활용하세요
  - 현재 패턴으로는 "0번부터 찍기" 전략이 유효할 수 있습니다

## 새로운 퀴즈 문제 추가 자동화

새 퀴즈를 추가할 때도 형식의 일관성과 가이드라인을 준수하는 것이 중요합니다. 이를 자동화하기 위해 카테고리과 난이도를 인수로 받는 커스텀 명령어 '/quiz-add'를 생성해 봅시다. 예를 들어, 한 국사 카테고리에 중급 난이도 퀴즈를 새로 추가하려면 '/quiz-add 한국사 중급'이라고 입력하면 됩니다.

▶ /quiz-add \$카테고리 \$난이도

**01** 새 퀴즈 템플릿을 자동 생성하고 검증 가이드라인을 포함하는 명령어 '/quiz-add'를 생성하기 위해 클로드 코드에 다음과 같이 입력합니다.

#### 프롬프트

.claude/commands/quiz-add.md를 생성해.

새 퀴즈를 추가하는 명령어를 만들어 줘.

\$1은 카테고리, \$2는 난이도로 받아서 처리해.

기존 퀴즈들과 같은 형식으로 만들고, 검증 가이드라인을 꼭 지키게 해 줘.

> .claude/commands/quiz-add.md를 생성해.

새 퀴즈를 추가하는 명령어를 만들어 줘.

\$1은 카테고리, \$2는 난이도로 받아서 처리해.

기존 퀴즈들과 같은 형식으로 만들고, 검증 가이드라인을 꼭 지키게 해 줘. █

ctrl-g to edit prompt in start

**note** 중간 과정을 확인하면서 계속 진행하세요.

**02** 새로운 명령어 'quiz-add'를 생성했습니다. 사용법과 예시를 확인하세요.

- 새로운 slash command `/quiz-add`를 생성했습니다!

생성된 명령어 기능

사용법:

`/quiz-add` [카테고리] [난이도]

예시:

`/quiz-add` 한국사 medium

`/quiz-add` 과학 hard

**03** '/exit' 명령어로 클로드 코드를 종료한 뒤 'claude' 명령어로 재시작하세요. 생성한 명령어 '/quiz-add'를 활용해 중급 한국사 퀴즈를 추가해 봅시다. 클로드 코드에 다음과 같이 입력하세요.

▶ `/quiz-add` 한국사 medium

> `/quiz-add` 한국사 medium

▶▶ accept edits on (shift+tab to cycle)

**04** '한국사' 카테고리에 새 퀴즈가 추가되었습니다.

-  문제 추가 완료!

새로운 한국사 문제(ID: 41)가 `quiz-data.js`에 성공적으로 추가되었습니다.

추가된 문제:

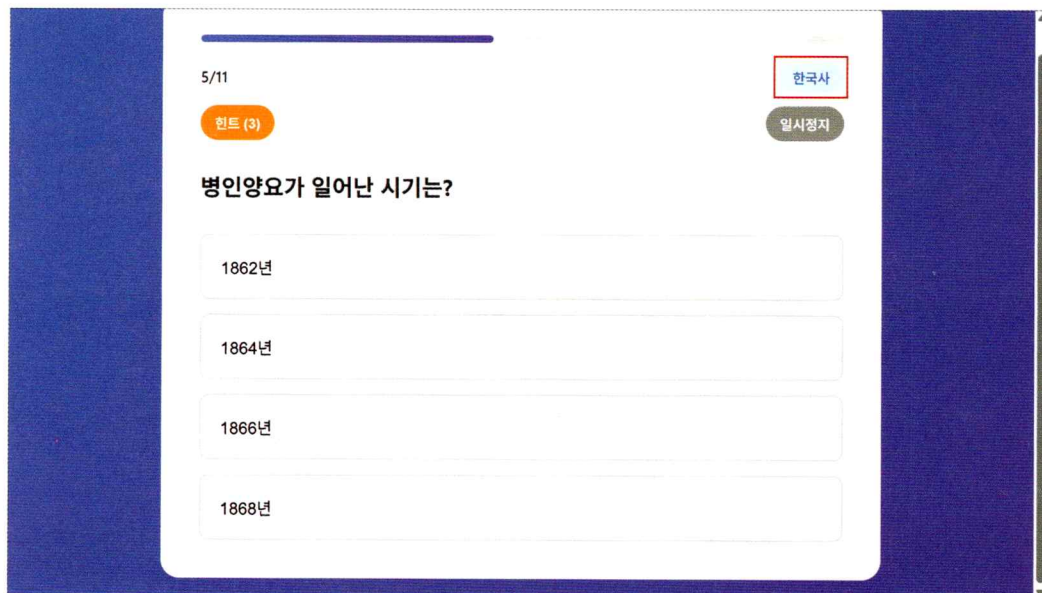
- 카테고리: 한국사
- 난이도: medium
- 문제: 병인양요가 일어난 시기는?
- 정답: 2번 - 1866년

개선 효과:

- 한국사 카테고리의 정답 위치 편향 개선 (2번 선택지 활용)
- 전체 문제 수: 40개 → 41개
- 한국사 문제 수: 10개 → 11개



**05** 새 퀴즈가 추가되었는지 확인하기 위해 데스크톱의 'Study-03' 폴더에서 'index.html' 파일을 더블 클릭해 퀴즈 게임을 실행해 보세요.



이번 절에서는 퀴즈 검증, 분석, 제작하는 작업을 커스텀 명령어로 자동화하여 개발 효율과 콘텐츠 품질을 함께 높이는 방법을 다루었습니다. '/quiz-validate' 명령어는 모호한 최상급 표현을 자동으로 검증하고, '/quiz-range' 명령어는 특정 범위의 퀴즈 난이도와 정답 분포를 분석하며, '/quiz-add' 명령어는 카테고리, 난이도별 새 문제를 일관된 가이드라인을 준수하며 추가할 수 있었습니다.

이처럼 인수를 활용한 커스텀 명령어는 반복되는 개발 작업을 간소화하는 강력한 도구입니다. 다음 절에서는 이렇게 만들어진 명령어를 프로젝트 업데이트와 자동화 워크플로에 연결하는 방법을 소개하겠습니다.

### ▶ 5가지 키워드로 끝내는 핵심 포인트

- **커스텀 명령어**는 사용자가 직접 정의해 반복 작업을 자동화할 수 있는 개인화된 명령어입니다.
- **전역 명령어**는 홈 디렉터리(~/.claude/commands/)에 정의되어 모든 프로젝트에서 공통으로 사용할 수 있습니다.
- **프로젝트 명령어**는 현재 작업 폴더(.claude/commands/)에 저장되어 해당 프로젝트에서만 적용됩니다.
- **인수**는 명령어나 함수에 전달되는 추가 정보로, 작업의 대상이나 범위를 지정하는 역할을 합니다.
- **플레이스홀더**는 \$ARGUMENTS, \$1, \$2처럼 인수가 들어갈 자리를 표시하는 기호로, 실행 시 실제 값으로 치환됩니다.

### ▶ 확인 문제

1. 다음 설명 중 틀린 것을 고르세요.

- ① 프로젝트 명령어는 해당 프로젝트에서만 사용 가능하다.
- ② 커스텀 명령어는 자연어로 작성할 수 있다.
- ③ 명령어 생성 후 즉시 사용 가능하다.
- ④ 마크다운 파일 형식으로 명령어를 저장한다.
- ⑤ 매개변수를 통해 유연한 동작이 가능하다.

**2.** 다음 빈칸에 들어갈 적절한 답을 쓰세요.

- 프로젝트 전용 커스텀 명령어가 저장되어야 할 위치는 (            ) 이다.
- 새 명령어 작성 후 사용가능하게 하려면 클로드 코드를 (            ) 해야 한다.
- 커스텀 명령어에서 특정 번호 범위의 문제만 검토하려면 (            ,            )와 같이 인수 플레이스 홀더를 사용한다.
- 커스텀 명령어에서 사용자 입력값을 받으려면 (            )와 같은 인수 플레이스 홀더를 사용한다.
- ~/.claude/commands에 만드는 명령어는 (            )에서 사용 가능하다.
- .claude/commands에 만드는 명령어는 (            )에서 사용 가능하다.

# 05-3

## 명령어 체이닝을 통해 익히는 유지보수 전략

핵심 키워드

유지보수

명령어 체이닝

통합 커스텀 명령어

이번 절에서는 플레이 기록을 안전하게 저장한 뒤 리더보드와 통계 대시보드를 구현해 봅니다. 여러 커스텀 명령어를 연결하거나 통합 명령어로 묶어, 문제 추가부터 검증, 보고서 생성까지 이어지는 완전 자동화된 게임 제작 워크플로를 구축합니다. 이를 통해 지속 가능한 AI 기반 퀴즈 게임 시스템을 완성하게 됩니다.

### 시작하기 전에

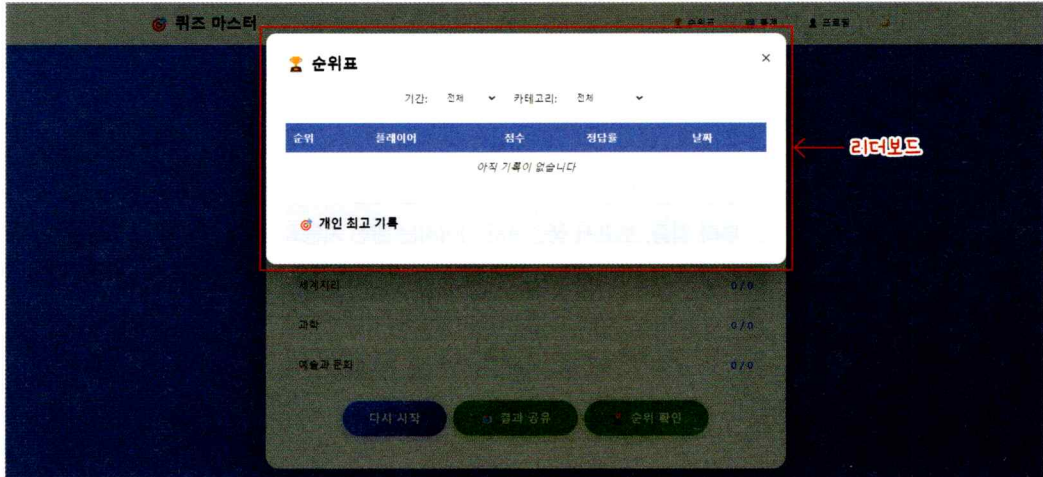
퀴즈 게임은 새로운 문제를 꾸준히 추가하고, 기존 문제의 오류를 수정하거나 난이도를 조정하는 등 **지속적인 업데이트**와 **유지보수**가 중요합니다. 하지만 이런 반복 작업을 매일 수동으로 하다 보면 금세 귀찮고 번거로워지죠.

다행히 AI는 이런 유지보수 과정도 자동화해 줍니다. 명령어 하나만 실행하면 문제 추가, 검증, 통계 분석, 백업까지 하루치 업데이트를 한 번에 처리할 수 있죠. AI가 관리 루틴을 대신 수행하도록 설계하면 개발자는 반복 작업 대신 새로운 아이디어와 콘텐츠 개선에 집중할 수 있습니다.



### 3단계: 데이터 저장 및 순위 시스템

3단계는 사용자 플레이 기록을 로컬에 안전하게 저장하고, 이를 바탕으로 게임에 리더보드 leader board, 순위표, 개인 통계, 대시보드를 구현해 봅니다.



**01** 클로드 코드에 앞서 만든 3단계 프롬프트를 입력합니다. 실행을 마치면 완료되었다는 메시지가 나타납니다.

✅ 3단계 완료: 데이터 저장 및 순위 시스템

🔗 구현된 기능

#### 1. LocalDataManager 클래스 (data-manager.js)

- ✅ 게임 결과 자동 저장 (localStorage)
- ✅ 통계 데이터 추적 및 업데이트
- ✅ 최근 100개 게임 기록 유지
- ✅ 카테고리별 통계 관리

#### 2. 순위 시스템

- ✅ 전체 / 주간 / 일일 리더보드
- ✅ 상위 10개 기록 표시

깃허브 자료실  
바로가기

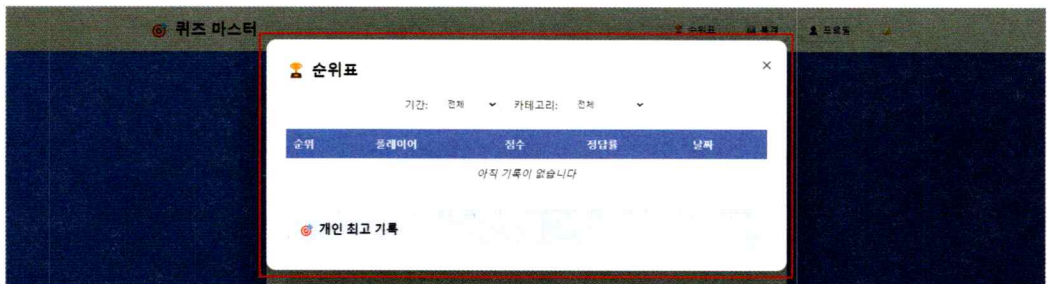
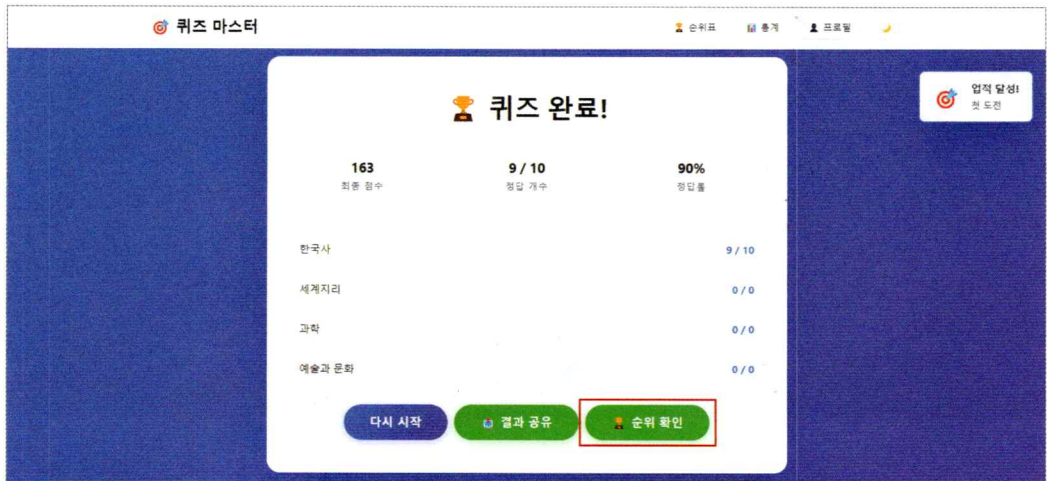


**note** 깃허브 자료실(<https://github.com/taehojo/vibecoding>)의 [전체 프롬프트 모음 보기]에 이 책에서 제시된 예시 프롬프트를 장별로 제공하고 있습니다.

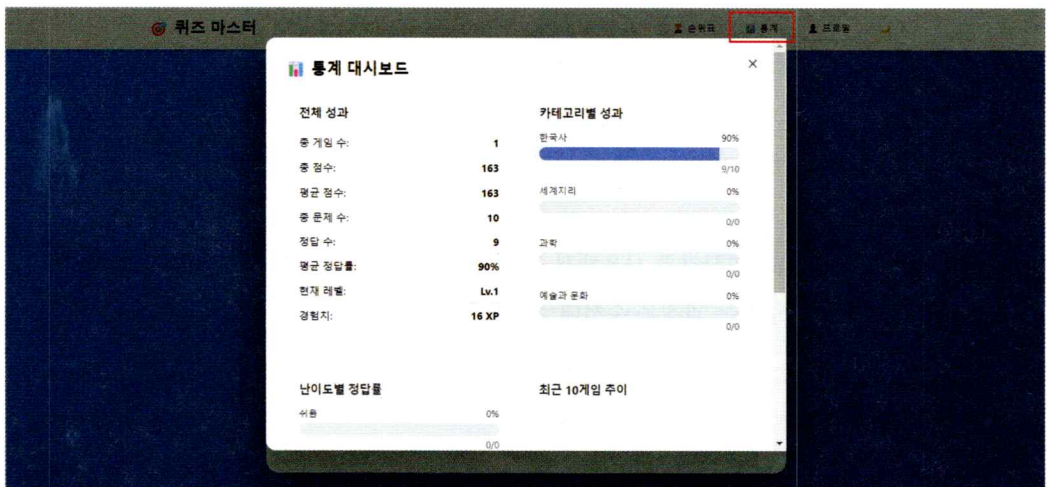
05-1절에서 생성한 단계별 프롬프트에서 3단계 프롬프트를 복사한 뒤, 클로드 코드에 붙여 넣으세요.



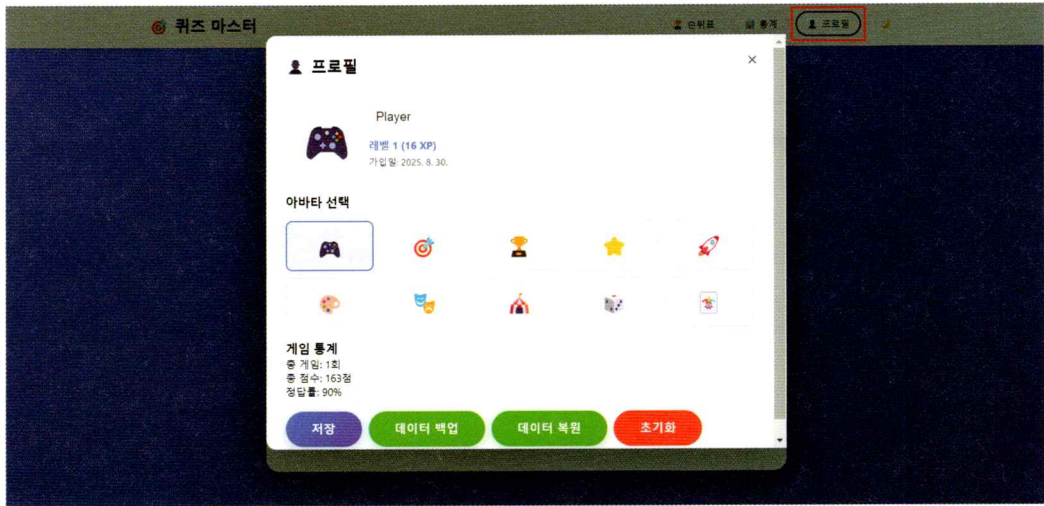
**02** 데스크톱의 'Study-03' 폴더에서 'index.html' 파일을 실행해 리더보드와 통계 대시보드를 확인해 봅시다. 퀴즈 게임을 완료하면 세부 통계를 보여주고, [순위 확인] 버튼을 누르면 리더보드가 나타납니다.



**03** 상단의 [통계] 버튼을 누르면 통계 대시보드도 확인할 수 있습니다.



04 상단의 [프로필] 버튼을 누르면 사용자 프로필을 편집할 수 있습니다.



## 게임 제작 워크플로 자동화하기

커스텀 명령어는 개별적으로도 유용하지만, 여러 명령어를 연결하여 더 강력한 자동화를 구현할 수 있습니다. 이제 게임을 제작하는 일련의 과정을 하나의 **파이프라인** pipeline 으로 묶어, ‘문제 추가 – 검증 – 통계 – 보고서 생성’의 단계를 자동으로 처리하도록 해 볼 것입니다.

### 명령어 체이닝

**명령어 체이닝** command chaining 은 여러 커스텀 명령어를 순서대로 연결 실행해서 반복적인 개발, 관리 작업을 자동화하는 방법입니다. 가장 간단한 형태는 && 연산자로 두 개 이상의 명령어를 순서대로 연결하는 방법입니다. 앞의 명령이 성공적으로 실행된 경우에만 다음 명령이 이어집니다.

```
▶ /명령어1 && /명령어2 && /명령어3
```

예를 들어, 다음과 같이 && 연산자를 사용해서 두 명령어를 체이닝하면 ‘한국사’ 카테고리에 새로운 상급 문제를 추가한 뒤 검증하는 과정을 순서대로 진행합니다.

```
▶ /quiz-add 한국사 상급 && /quiz-validate 한국사
```



&& 연산자를 사용해서 명령어를 체이닝할 때 주의할 점은 앞 명령이 실패하면 다음 명령이 실행되지 않는다는 점입니다. 따라서 새로운 체이닝을 만들 때는 작은 단위로 개별 테스트한 후 연결해야 합니다.

## 퀴즈 생성, 분석, 점수 집계하기

이번에는 여러 개의 커스텀 명령어를 생성하고 체이닝해서 게임 제작 과정을 효율적으로 만들어 봅시다. 생성할 커스텀 명령어는 세 가지입니다. 다음 세 명령어는 퀴즈 시스템의 정확성을 자동으로 유지, 관리하는 핵심 관리 도구 세트가 될 것입니다.

- **/quiz-check**: 모든 퀴즈를 대상으로 정답이 올바른지 자동으로 검증하는 명령어
- **/quiz-stats**: 퀴즈 게임의 전체 데이터를 분석해 진행 현황과 난이도 분포를 수치화하는 명령어
- **/quiz-leaderboard**: 플레이어의 점수를 집계하고 리더보드를 자동으로 갱신하는 명령어

**01** 다음과 같이 필요한 명령어들을 생성할 수 있도록 클로드 코드에 한 번에 명령합니다.

### 프롬프트

```
.claude/commands/quiz-check.md 를 생성해 줘.
여기에는 모든 문제 정답의 정확성을 검증하는 기능을 수행하게 해.

.claude/commands/quiz-stats.md 를 만들어 줘.
퀴즈 게임의 통계를 관리하는 기능을 하게 해 줘.

.claude/commands/quiz-leaderboard.md 를 만들어.
순위 시스템을 관리하는 기능이 실행되게 해.
```

```
> .claude/commands/quiz-check.md 를 생성해 줘.
여기에는 모든 문제 정답의 정확성을 검증하는 기능을
수행하게 해.
```

```
.claude/commands/quiz-stats.md 를 만들어 줘.
퀴즈 게임의 통계를 관리하는 기능을 하게 해 줘.
```

```
.claude/commands/quiz-leaderboard.md 를 만들어.
순위 시스템을 관리하는 기능이 실행되게 해.█
```

```
▸▸ accept edits on ·1 background ctrl-g to edit
(shift+tab to cycle) task prompt in start
```



## 02 세 가지 커스텀 명령어가 생성된 것을 확인합니다.

- Perfect! 세 개의 새로운 slash command를 생성했습니다.

✓ 생성된 명령어

1. /quiz-check - 문제 정답 검증

파일: .claude/commands/quiz-check.md

기능:

- ✓ 모든 문제의 구조적 검증
- ✓ 정답 정확성 확인
- ✓ 설명의 적절성 검토
- ✓ 의심스러운 문제 식별
- ✓ 오류 발견 시 즉시 보고

## 03 '/exit' 명령어와 'claude' 명령어로 클로드를 재시작하세요. 그리고 나서 '/help' 명령어를 통해 새로운 명령어가 잘 생성되었는지 확인합니다.



Claude Code v2.0.13  
Sonnet 4.5 · Claude Pro  
C:\Users\user\Desktop\VibeCoding\Study-03

> /help

— Claude Code — ———— custom-command (tab to —  
v2.0.13 generalcommandsands cycle)  
Browse custom commands:  
> /quiz-check 모든 퀴즈 문제의 정답 정확성을  
For more help: 검증합니다 (project)  
> /quiz-leaderboard 퀴즈 게임의 순위 시스템을  
For more help: 관리합니다 (project)  
> /quiz-stats 퀴즈 게임의 통계를 분석하고  
For more help: 관리합니다 (project)

**note** '/help' 명령어에서 빠져나올 때는 [Esc] 키를 누르세요.

## 04 '/quiz-add' 명령어로 '한국사' 카테고리에 상급 난이도 퀴즈를 추가하고, '/quiz-check' 명령어와 '/quiz-stats' 명령어를 체이닝 하여 정확성을 검증과 통계 분석까지 이어서 실행하도록 해 봅시다. 클로드 코드에 다음과 같이 명령어를 입력하세요.

▶ /quiz-add 한국사 상급 && /quiz-check && /quiz-stats

> /quiz-add 한국사 상급 && /quiz-check &&  
/quiz-stats

**05** ‘한국사’ 카테고리에 새로운 상급 문제를 추가했습니다. ‘/quiz-check’ 명령어가 전체 문제들을 점검해서 정상 작동하고 중복이 없음을 확인하고 동시에 문제 난이도를 정리했습니다. 그리고 나서 ‘/quiz-stats’ 명령어가 보기 편한 문제를 확인해 주었습니다.

```
● ☒ 문제가 성공적으로 추가되었습니다!

이제 명령어 인자에서 요청하신 대로 /quiz-check와
/quiz-stats를 실행하겠습니다.

> /quiz-check is running...

● 퀴즈 데이터를 체계적으로 검증하겠습니다.

□ 구조적 검증

quiz-data.js 파일을 분석한 결과:

☒ 구조 검증 통과:
```

## 통합 커스텀 명령어로 일일 업데이트 한 번에 끝내기

이번에는 여러 개의 커스텀 명령어를 한 덩어리로 묶어 매일 해야 하는 업데이트, 검증, 백업, 보고를 한 번에 자동으로 수행하는 **통합 커스텀 명령어**를 만들어 보겠습니다. 명령어 체이닝을 사용하는 것도 좋지만, 실제 프로젝트에서는 통합 커스텀 명령어가 더 실용적이죠.

퀴즈 게임에 매번 같은 퀴즈만 출제되면 플레이어는 금방 흥미를 잃어버릴 것입니다. 따라서 매일 업데이트해야 하죠. 지금부터 생성할 통합 커스텀 명령어 ‘/quiz-daily’는 ❶ 퀴즈 데이터 구조 파악 → ❷ 문항 수·분포 확인 → ❸ 카테고리 결손 파악 → ❹ 추가 전 중복 체크 → ❺ 새 문제 생성 후 형식·규칙 검증 → ❻ 전체 데이터 백업 → ❼ 결과 리포트 출력까지 파이프라인으로 순차 실행합니다. 각 단계마다 검증하고 오류를 체크하므로 실패 시 즉시 중단·오류 보고를 하고, 성공하면 변경사항이 안전하게 반영됩니다.

통합 커스텀 명령어를 생성할 때는 몇 가지 주의사항을 따라야 합니다. 다음 조건을 유지하면서, 할루시네이션 없이 새로운 문제를 매번 업데이트하고 결과를 점검하는 통합 코드를 만들어 보겠습니다.

1. 파일 구조를 확인하게 할 것.
2. 작은 단위로 테스트해 볼 것.
3. 복잡한 작업은 단계별로 나누어 검증할 것.

**01** 통합 커스텀 명령어 '/quiz-daily'를 생성하기 위해 클로드 코드에 다음과 같이 입력하세요.

**프롬프트**

.claude/commands/quiz-daily.md를 만들어서 다음 작업을 순서대로 수행하게 해 줘.

1. 퀴즈가 담겨 있는 파일의 구조를 읽고 이해하기
2. 현재 문제 개수와 분포 확인
3. 각 카테고리별로 부족한 부분 파악
4. 새 문제 추가 전 중복 체크
5. 문제 추가 후 형식 검증
6. 전체 데이터 백업
7. 실행 결과 상세 보고

각 단계별로 검증하고, 실패하면 즉시 중단한 뒤, 오류를 보고해.

> .claude/commands/quiz-daily.md를 만들어서 다음 작업을 순서대로 수행하게 해 줘.

1. 퀴즈가 담겨 있는 파일의 구조를 읽고 이해하기
2. 현재 문제 개수와 분포 확인
3. 각 카테고리별로 부족한 부분 파악
4. 새 문제 추가 전 중복 체크
5. 문제 추가 후 형식 검증
6. 전체 데이터 백업
7. 실행 결과 상세 보고

각 단계별로 검증하고, 실패하면 즉시 중단한 뒤, 오류를 보고해.■

**02** 실행이 완료되면 '/exit' 명령어와 'claude' 명령어로 클로드 코드를 재시작합니다.

- 완료했습니다! .claude/commands/quiz-daily.md 파일을 생성했습니다.

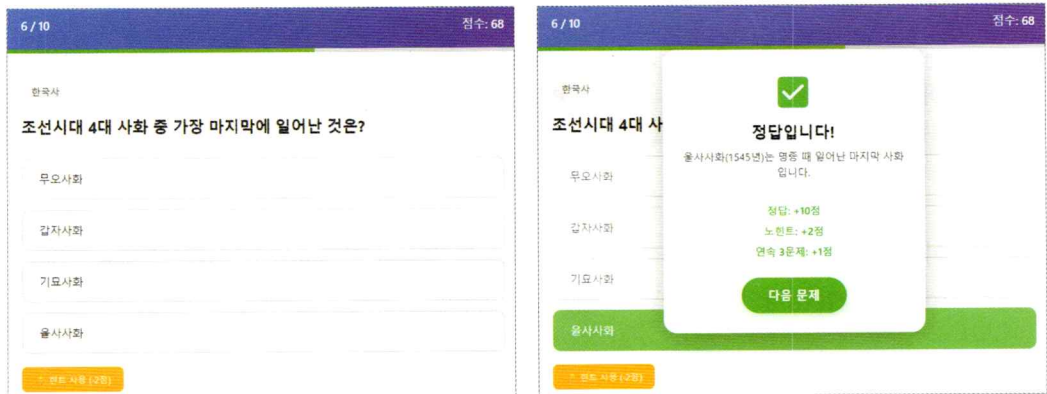
✅ 생성 완료

📄 명령어 기능

/quiz-daily 명령어는 다음 작업을 순서대로 수행합니다:

1. 퀴즈 데이터 구조 분석 - quiz-data.js 읽고 구조 파악
2. 현재 문제 개수와 분포 확인 - 카테고리별, 난이도별 집계
3. 부족한 부분 파악 - 개선이 필요한 카테고리

**03** ‘Study-03’ 폴더에서 ‘index.html’을 실행해 ‘/quiz-daily’ 명령어로 일일 업데이트를 진행한 결과를 확인하세요. 새로운 문제로 업데이트된 것을 확인할 수 있습니다.



## 클로드 코드가 스스로 만드는 커스텀 명령어

지금까지는 플레이어의 입장에서 문제를 풀고 성적을 확인하는 데 집중했습니다. 이제 플레이어의 성적을 보고 관리하는 **선생님 모드**를 추가해 보겠습니다. ‘선생님 모드’는 여러 플레이어의 퀴즈 결과를 자동으로 수집, 분석, 비교하고 보고서로 시각화하는 통합 관리 시스템입니다.

이번에는 어떤 커스텀 명령어가 필요한지 스스로 판단해 만들고, 명령어를 통합하여 자동화하는 과정까지 모두 클로드 코드에게 맡겨 보겠습니다.

**01** 다음과 같이 입력하여 선생님 모드를 위한 커스텀 명령어들을 생성합니다.

### 프롬프트

이제 퀴즈를 푼 여러 학생들의 성적을 한눈에 보고 비교하는 선생님 모드를 만들고 싶어.  
이 기능에 필요한 커스텀 명령들을 직접 구상하고 만들어 줘.  
그리고 이 명령어를 하나로 모아서 실행하는 통합 명령어도 만들어 줘.  
모든 명령들은 ‘.claude/commands/’ 폴더 안에 각각 .md 확장자로 저장되어야 해.  
실행이 끝나면, 생성한 커스텀 명령어들과 그 기능을 나열해서 보고해.

> 이제 퀴즈를 푼 여러 학생들의 성적을 한눈에 보고 비교하는 선생님 모드를 만들고 싶어.  
이 기능에 필요한 커스텀 명령들을 직접 구상하고 만들어 줘.  
그리고 이 명령어를 하나로 모아서 실행하는 통합 명령어도 만들어 줘.  
모든 명령들은 ‘.claude/commands/’ 폴더 안에 각각 .md 확장자로 저장되어야 해.  
실행이 끝나면, 생성한 커스텀 명령어들과 그 기능을 나열해서 보고해.■



## + 여기서 잠깐

## 명령어를 문서로 남기는 이유

커스텀 명령어를 생성할 때는 각 명령어가 '.claude/commands/' 폴더 안에 각각 .md 파일로 저장하는 것이 중요합니다.

이렇게 하면 명령어의 동작 방식과 수정 내역이 모두 문서 형태로 기록되어 나중에 특정 기능만 업데이트하거나 개선할 때 해당 명령어 파일만 수정하면 됩니다. 결과적으로 전체 프로젝트를 유지하면서도 일부 기능만 안전하게 교체 및 업데이트 할 수 있어 유지보수와 버전 관리가 훨씬 효율적입니다.

**02** 실행이 완료되면 클로드 코드가 보여주는 보고서를 꼼꼼히 확인하세요. 여기서는 '/teacher-dashboard'라는 통합 명령어로 선생님 모드를 자동 실행할 수 있다고 합니다.

**\*\*기능 : \*\***

- 완료되었습니다! 선생님 모드를 위한 커스텀 명령어 시스템을 성공적으로 구축했습니다.

 통합 명령어

6. **/teacher-dashboard** ★ (권장)
  - 위의 모든 명령어를 한 번에 실행
  - 종합 리포트 생성
  - 학급 전체 강점/약점 및 교육 제안사항 제공

파일 위치

모든 명령어는 **.claude/commands/** 폴더에 **.md**

**note** 생성형 AI의 특성상 각자의 실행 결과는 책과 다를 수 있습니다.

**03** '/exit' 명령어와 'claude' 명령어를 실행해 클로드 코드를 재실행한 뒤, 새로 생성한 통합 커스텀 명령어 '/teacher-dashboard'를 입력하세요.



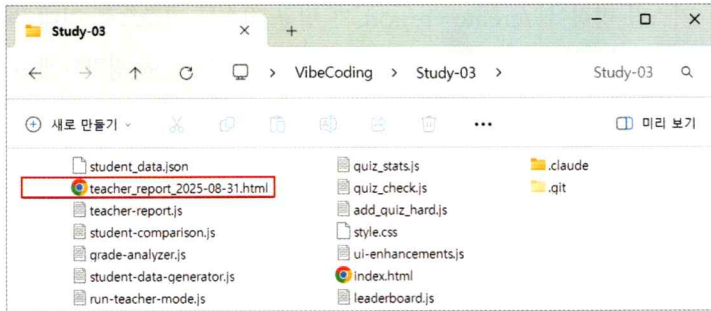
Claude Code v2.0.13  
Sonnet 4.5 · Claude Pro  
C:\Users\user\Desktop\VibeCoding\Study...

> /teacher-dashboard

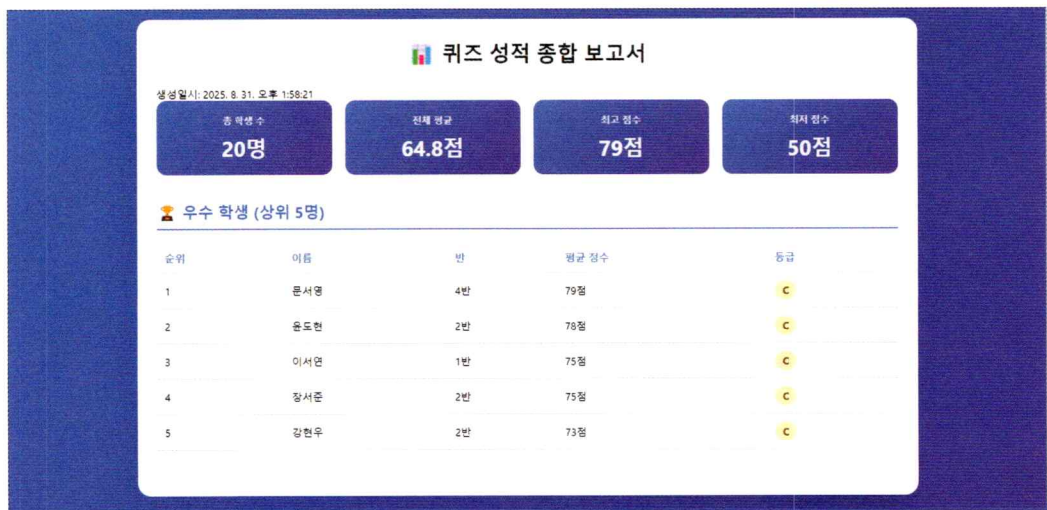
/teacher-dashboard  
선생님 통합 대시보드 (project)

/teacher-leaderboard  
우수 학생 순위 (리더보드) (project)

**04** 실행이 완료되면 'Study-03' 폴더에 생성된 선생님 모드 대시보드 파일 'teacher\_report.html'을 확인해 봅니다.



**05** 선생님을 위한 퀴즈 성적 종합 보고서 페이지를 만들었습니다.



## 효율적인 유지보수 워크플로 구축하기

AI로 개발을 하다 보면 특정 부분만 수정하고 싶은데도 전체를 다시 만들어야 하는 상황이 자주 발생합니다. 이는 시스템이 기능별로 분리되지 않고 하나로 통합되어 있기 때문입니다. 커스텀 명령어를 각각 독립된 .md 파일로 관리해 두면, 필요한 부분만 선택적으로 업데이트하면서도 전체 구조는 그대로 유지한 채 효율적으로 개선할 수 있습니다.

## 기존 커스텀 명령만 업데이트하기

선생님 모드에서 보여준 대시보드에는 ‘등급’ 열이 있는데, 모든 플레이어의 등급이 ‘C’로 분류되어 있습니다. 이는 등급을 분류하는 커스텀 명령어 ‘/create-report’가 평균 점수를 기준으로 절대 평가했기 때문입니다. 커스텀 명령어 ‘/create-report’가 플레이어의 등급을 상대적으로 평가하게 수정해 봅시다.

**01** 커스텀 명령어 ‘/create-report’를 수정하기 위해 클로드 코드에 다음과 같이 입력하세요.

### 프롬프트

.claude/commands/create-report.md를 다음과 같이 수정해 줘.

등급 표시를 상위 비율 기준 상대평가로 변경해서 다음과 같이 표시되게 해.

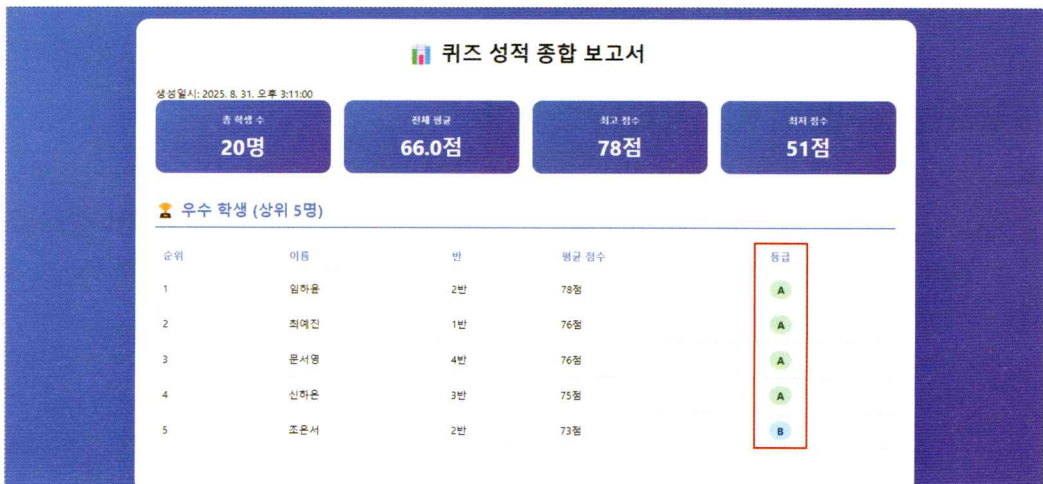
- 상위 20%: A
- 상위 40%: B
- 상위 70%: C
- 하위 30%: D

```
> .claude/commands/create-report.md를 다음과 같이 수정해 줘.
등급 표시를 상위 비율 기준 상대평가로 변경해서 다음과 같이 표시되게 해.

- 상위 20%: A
- 상위 40%: B
- 상위 70%: C
- 하위 30%: D

>> accept edits on (shift+tab to cycle) ctrl-g to edit prompt in start
```

**02** 클로드 코드를 재시작하고, ‘/create-report’를 실행해 플레이어의 등급을 재평가합니다.



## 선생님 모드 보고서 파일로 내보내기

전체 명령어에 새로운 커스텀 명령어를 추가해서 일부 기능만 확장하는 것도 가능합니다. 이번에는 대시보드 데이터를 외부 파일로 내보내는 커스텀 명령어를 추가해 봅시다.

**01** 선생님 모드로 생성한 대시보드를 파일로 내보내는 새로운 커스텀 명령어 `/export-report`를 생성하기 위해 클로드 코드에 다음과 같이 입력합니다.

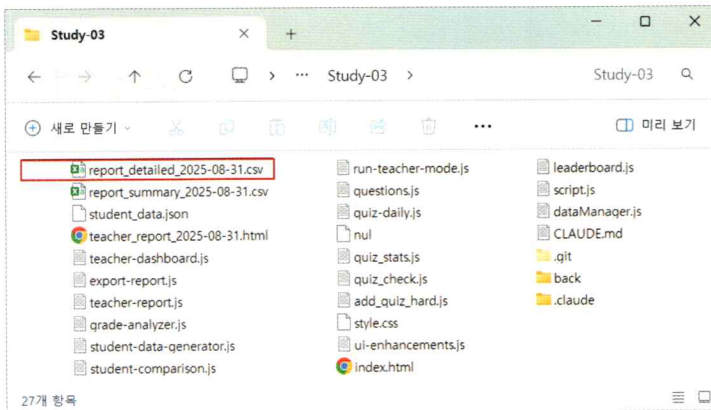
### 프롬프트

```
.claude/commands/export-report.md 파일을 새로 만들어 줘.
teacher_report.html을 읽어서 CSV 또는 PDF로 저장하고,
teacher-dashboard.md에 이 명령어를 추가해.
```

```
> .claude/commands/export-report.md 파일을 새로 만들어 줘.
teacher_report.html을 읽어서 CSV 또는 PDF로 저장하고,
teacher-dashboard.md에 이 명령어를 추가해. █
```

```
▶▶ accept edits on (shift+tab to ctrl-g to edit prompt in
cycle) start
```

**02** 새로운 명령어 `/export-report`를 생성하고 나서, 클로드 코드를 재시작한 뒤, `/teacher-dashboard` 명령어를 실행합니다. `Study-03` 폴더에 새로운 파일 `report_detailed.csv`가 생성되었습니다.





**03** CSV 파일을 더블 클릭해 열어 보면 학생들의 평가 보고서가 정리되어 있는 것을 확인할 수 있습니다.

| 학생ID   | 이름  | 반  | 평균점수 | 등급 | 퀴즈횟수 |
|--------|-----|----|------|----|------|
| STU009 | 임하운 | 2반 | 78   | A  | 8    |
| STU005 | 최예진 | 1반 | 76   | A  | 5    |
| STU020 | 문서경 | 4반 | 76   | A  | 5    |
| STU015 | 신하은 | 3반 | 75   | A  | 6    |
| STU007 | 조은서 | 2반 | 73   | B  | 7    |
| STU011 | 권지우 | 3반 | 72   | B  | 9    |
| STU013 | 오유진 | 3반 | 72   | B  | 9    |
| STU008 | 윤도현 | 2반 | 71   | B  | 9    |
| STU012 | 한승민 | 3반 | 68   | C  | 10   |
| STU016 | 배준서 | 4반 | 67   | C  | 6    |
| STU002 | 이서연 | 1반 | 66   | C  | 9    |
| STU003 | 박지호 | 1반 | 64   | C  | 7    |
| STU017 | 송다은 | 4반 | 63   | C  | 9    |
| STU006 | 강현우 | 2반 | 62   | C  | 5    |
| STU019 | 홍지민 | 4반 | 62   | D  | 9    |
| STU004 | 정수민 | 1반 | 61   | D  | 10   |
| STU010 | 장서준 | 2반 | 59   | D  | 8    |
| STU001 | 김민준 | 1반 | 52   | D  | 6    |
| STU014 | 서민재 | 3반 | 51   | D  | 8    |
| STU018 | 양현서 | 4반 | 51   | D  | 7    |

이번 절에서는 퀴즈 게임에 데이터 저장, 통계, 리더보드 시스템을 구현하고 이를 효율적으로 관리 하기 위한 자동화 워크플로를 구축했습니다. 여러 커스텀 명령어를 체이닝하여 문제 추가 → 검증 → 통계 분석 → 보고서 생성 과정을 자동화하고, 나아가 통합 명령어로 일일 업데이트를 한 번에 처리 하는 방법을 익혔습니다.

또한 '선생님 모드'를 통해 여러 플레이어의 성적을 자동으로 수집 및 비교하여 대시보드를 만들었습니다. 이때 클로드 코드에게 스스로 필요한 명령어를 생성하도록 했고, 일부 기능만 수정하거나 확장 할 수 있는 유연한 업데이트 구조도 경험했습니다. 이처럼 커스텀 명령어와 통합 자동화를 활용하면 반복 작업을 줄이고, AI가 스스로 관리 및 검증하는 지속 가능한 개발 환경을 구축할 수 있습니다.

### ▶ 3가지 키워드로 끝내는 핵심 포인트

- **유지보수**는 완성된 프로젝트를 꾸준히 관리하고 개선하는 과정으로, 기능을 모듈화해 두면 필요한 부분만 선택적으로 수정하면서 전체 구조는 그대로 유지할 수 있습니다.
- **명령어 체이닝**은 연산자 **&&**를 사용해 여러 커스텀 명령어를 순서대로 연결 실행해 반복 작업을 자동화하는 방법입니다.
- **통합 커스텀 명령어**는 여러 작업을 하나의 명령어로 묶어 한 번에 처리할 수 있게 해 주며, AI가 스스로 관리, 운영하는 지속 가능한 개발 환경을 만들어 줍니다.

### ▶ 확인 문제

1. 다음 설명 중 틀린 것을 고르세요.

- ① && 연산자로 연결된 명령어는 순차적으로 실행된다.
- ② 첫 번째 명령이 실패해도 두 번째 명령은 실행된다.
- ③ 통합 명령어로 여러 개별 명령어를 한 번에 실행할 수 있다.
- ④ 개별 명령어 파일만 수정하여 부분 업데이트가 가능하다.
- ⑤ 명령어를 모듈화하면 유지보수가 용이하다.

2. 다음 빈칸을 채우세요.

- 두 명령어를 연결하여 순차 실행하려면 (            ) 연산자를 사용한다.
- 새 기능을 추가할 때는 새로운 (            )를 만들고 통합 명령어에 추가한다.
- 커스텀 명령어는 (            ) 파일 형식으로 저장한다.